数据结构

Morning Flower and Evening Oath

目录

第一章 哈希表 **1**

1.1 GNU gp_hash_table 与 cc_hash_table . . . 1

第二章 并查集 **3**

2.1 并查集 . . . 3

2.1.1 并查集 . . . 3

2.1.2 可撤销并查集 . . . 5

2.1.3 带势并查集与差值约束 . . . 7

2.1.4 时间线段树分治：离线动态连通性 . . . 9

2.1.5 并查集模板题的接口范围 . . . 12

第三章 堆 **13**

3.1 配对堆 . . . 13

3.1.1 PBDS 可修改、可合并优先队列 . . . 13

3.2 左偏树 . . . 15

3.2.1 左偏树可并小根堆 . . . 15

第四章 ST 表 **17**

4.1 位掩码静态区间最值与位置 . . . 17

第五章 树状数组 **21**

5.1 四矩二维树状数组：矩形加与矩形和 . . . 21

5.2 树状数组第 k 小与组合依赖 . . . 23

5.3 离线静态矩形权值和 . . . 24

第六章 线段树 **28**

6.1 线段树基础 . . . 28

6.1.1 通用懒标记线段树与边界搜索 . . . 28

6.1.2 通用线段树与左右边界搜索 . . . 34

6.1.3 线段树维护单调栈：区间加与接雨水 . . . 39

6.1.4 ACL 的组合约定 . . . 41

6.1.5 线段树维护单调栈：双向扫描与接雨水 . . . 42

6.2 线段树合并 & 分裂 . . . 42

6.2.1 线段树合并与分裂 . . . 42

6.3 李超线段树 . . . 51

6.3.1 精确线段李超树：最大值编号 . . . 51

6.3.2 离散李超树与最优直线编号 . . . 56

6.4 区间最值操作 & 区间历史最值 . . . 59

6.4.1 Segment Tree Beats：区间截断、加法与和 . . . 59

第七章 二叉搜索树 & 平衡树 **65**

7.1 二叉搜索树 & 平衡树 . . . 65

7.1.1 PBDS 有序树与顺序统计 . . . 65

7.1.2 GNU rope 与版本共享 . . . 67

7.2 Treap . . . 68
7.2.1 FHQ Treap：可重集合与顺序统计 . . . 68
7.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和 . . . 72
7.2.3 隐式 Treap 位翻转与顺序反转 . . . 77
7.3 Splay 树 . . . 78
7.3.1 Splay：可重集合与顺序统计 . . . 78
7.4 替罪羊树 . . . 84
7.4.1 替罪羊树：重复计数与墓碑清理 . . . 84
7.5 笛卡尔树 . . . 85
7.5.1 笛卡尔树与相等值优先级 . . . 85

第八章 可持久化数据结构 87
8.1 可持久化线段树 . . . 87
8.1.1 主席树：静态区间第 k 小 . . . 87
8.1.2 可持久化区间加与版本拼接 . . . 89
8.1.3 可持久化并查集 . . . 92
8.1.4 可持久化区间拼接的懒标记 . . . 95
8.1.5 杭州 2023 K：跨版本拼接应用 . . . 95
8.2 可持久化平衡树 . . . 96
8.2.1 可持久化有序 Treap：分叉版本与顺序统计 . . . 96
8.3 可持久化字典树 . . . 102
8.3.1 前缀可持久化 01 字典树 . . . 102

第九章 树套树 104
9.1 树状数组套权值线段树 . . . 104
9.1.1 动态区间第 k 小（树状数组套权值线段树） . . . 104

第十章 K-D Tree 108
10.1 动态二维 K-D Tree：单点累加与矩形和 . . . 108
10.2 静态 K-D Tree 最近与最远点 . . . 111
10.3 动态 K-D Tree：矩形和与重建边界 . . . 113

第十一章 动态树 114
11.1 Link Cut Tree . . . 114
11.1.1 Link-cut tree：动态森林与路径异或 . . . 114
11.1.2 动态树路径乘积：虚子树与双向信息 . . . 119

第一部分 跨类依赖 120

附录 A 数论 121
A.1 模算术简介 . . . 121
A.1.1 动态模整数与合数模逆元 . . . 121
A.1.2 静态模整数 . . . 125

第一章　哈希表

1.1　GNU gp_hash_table 与 cc_hash_table

功能：整数键的哈希表：gp_map 是 GNU pb_ds 的开放寻址表，cc_map 是链式表，用法与 unordered_map 基本相同（下标访问、查找、插入、删除）。

核心思路：两者只是别名，关键在于自定义哈希 hash64：先给键加上程序启动时由时钟产生的随机种子，再做 splitmix64 的三轮异或移位与乘法混合，使高低位充分扩散，避免针对固定哈希函数构造的冲突数据卡掉表。

接口契约：GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表，cc_map<K,V> 为碰撞链接哈希表；K 为不超过 64 位的整数，复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty；insert 不覆盖已有键，operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1)，最坏 O(n)，空间 O(n)，不支持排名或有序前驱。纯键值查改可先试 gp_map；需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map，其节点分配和链接有成本，速度仍须按题实测。gp_map 修改后重新 find；cc_map 的点迭代器不等于可递增的遍历迭代器，删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize，也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子，无符号回绕；只针对整数键，不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map，cc_map 另行验证。

需要排名与有序前驱时，PBDS 平衡树见第 65 页。

```
struct hash64
{
    static uint64_t mix(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};

template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

模板题使用：Library Checker associative_array

题意：维护整数键值映射，修改为覆盖赋值，查询不存在的键输出 0。gp_map 用 operator[] 写入，用 find 查询，避免查询插入新键。所需条目中包含 hash64。

所需模板：gp_map（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    gp_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

第二章　并查集

2.1　并查集

2.1.1　并查集

功能：维护 n 个点的不相交集合，支持合并两个集合、判断两点是否同集合、查询集合大小以及列出所有集合。

核心思路：每个集合是一棵以根为代表的树，`fa` 存父亲，`sz` 在根处存集合大小。`find` 用路径减半：沿父链上走时把每个点的父亲改成祖父。`merge` 找到两根后把 y 的根接到 x 的根下并累加大小。`groups` 按根把点分桶，再删去空桶。

接口契约：dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 3 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

2.1.2 可撤销并查集

功能：支持撤销的并查集：合并两个集合、查询代表元，并能把结构恢复到之前某个快照时的状态，常用于离线版本树 DFS 或线段树分治。

核心思路：不做路径压缩，只按集合大小把小根接到大根下，树高为对数级，因此每次成功合并只改动一个 `fa[y]` 和一个 `siz[x]`。合并时把被接上的根 y 与 x 合并前的大小压入栈 `his`；快照就是栈长，`rollback(t)` 不断弹栈，恢复 `siz[fa[y]]` 并令 y 重新成为根。

接口契约：RollbackDSU(n) 可使用 0..n（若题目 0-based 则不用 n）。find 不压缩路径，merge 按集合大小合并，已同集合返回 false 且不记历史；siz 仅根处有效。snapshot() 返回历史长度，退出递归分支时 rollback(t) 撤销此后成功合并。t 必须是当前历史的祖先快照，不能用已回滚分支的旧快照恢复版本。find/merge 为 O(log(n+1))，每次撤销成功合并 O(1)，空间 O(n)；适合版本树或时间分治。它按大小合并以保证回滚复杂度，接口与普通 dsu 的根连接方向不同。

```
struct RollbackDSU
{
    vector<int> fa, siz;
    vector<pair<int, int>> his;

    RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x) const
    {
        while (x != fa[x]) x = fa[x];
        return x;
    }

    int snapshot() const { return (int)his.size(); }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        his.push_back({y, siz[x]});
        fa[y] = x;
        siz[x] += siz[y];
        return true;
    }

    void rollback(int t)
    {
        assert(0 <= t && t <= snapshot());
        while (snapshot() > t)
        {
            auto [y, s] = his.back();
            his.pop_back();
            siz[fa[y]] = s;
            fa[y] = y;
        }
    }
};
```

模板题使用：Library Checker persistent_unionfind

题意：离线读入所有操作，按所依赖的版本建立树。版本 -1 映射到树根 0，操作 i 映射到 i+1；顶点仍是 0-based。递归进入版本前保存 snapshot，退出时 rollback，答案按原查询顺序输出。只有合并产生可引用版本；本例是离线解法，不提供在线任意版本访问。总时间 O(n+q log(n+1))，空间 O(n+q)，版本链递归深度可为 q。

所需模板：RollbackDSU（第 5 页，2.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_unionfind

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> type(q + 1), u(q + 1), v(q + 1), ans(q + 1);
    vector<vector<int>> g(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int k;
        cin >> type[i] >> k >> u[i] >> v[i];
        g[k + 1].push_back(i);
    }
    RollbackDSU d(n);
    auto dfs = [&](auto &&self, int x) -> void
    {
        int t = d.snapshot();
        if (x)
        {
            if (type[x] == 0)
                d.merge(u[x], v[x]);
            else
                ans[x] = d.find(u[x]) == d.find(v[x]);
        }
        for (int y : g[x]) self(self, y);
        d.rollback(t);
    };
    dfs(dfs, 0);
    for (int i = 1; i <= q; i++)
        if (type[i] == 1) cout << ans[i] << '\n';
}
```

2.1.3 带势并查集与差值约束

功能：带权（带势）并查集：逐条加入差值约束 $p_u - p_v = w$，判断它与已接受的约束是否矛盾；并查询两点的差值 $p_u - p_v$（不连通时未知）。

核心思路：`d[u]` 存 $p_u - p_{\texttt{fa[u]}}$。`find` 递归到根后路径压缩，同时把父亲到根的差值累加进 `d[u]`，于是 `find` 后 `d[u]` 就是 $p_u - p_{\text{根}}$。加入约束时若 u, v 同根，只需检查 `d[u]` − `d[v]` $= w$；否则设两根为 a, b，由 $p_u - p_v = w$ 得 $p_b - p_a =$ `d[u]` − `d[v]` $- w$，把 b 挂到 a 下并存这个差（按大小交换方向时取相反数）。查询答案即 `d[u]` − `d[v]`。

接口契约：PotentialDSU<T>(n)，n≥0，点号 0..n−1。merge(u,v,w) 尝试添加 potential[u]−potential[v]=w，返回是否与已接受信息一致；重复一致约束也返回 true，矛盾返回 false，已有逻辑关系不变（仍可能路径压缩）。diff(u,v) 返回 optional<T>，未连通为 nullopt，连通时为 u−v；合法零/负值不是失败标志。find 求根并压缩路径，size 返回连通块大小。T 须提供精确加法交换群的 +、−、== 及零 T(0)，不是非交换群或浮点近似比较；整数须保证所有中间运算不溢出，动态模数不可在实例存活时改变。n 点、每条输入差值绝对值 ≤W 时，(2n+1)W≤LLONG_MAX 是 long long 的一个保守充分条件。按大小合并，摊还 O(α(n)) 次 T 运算、O(n) 空间；递归 find 未压缩前深度 O(log n)。fa/sz/d 为内部状态，不直接改写。原普通并查集的指定根合并方向与 bool 语义不变。

```
template <class T = long long> struct PotentialDSU
{
    vector<int> fa, sz;
    vector<T> d;

    PotentialDSU(int n) : fa(n), sz(n, 1), d(n, T(0))
    {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int u)
    {
        assert(0 <= u && u < (int)fa.size());
        int p = fa[u];
        if (p != u)
        {
            fa[u] = find(p);
            d[u] = d[u] + d[p];
        }
        return fa[u];
    }
```

约束合并、差值查询与分量大小（接上页同一结构体）：

```
    // Accept potential[u] - potential[v] = w iff it is consistent.
    bool merge(int u, int v, T w)
    {
        int a = find(u), b = find(v);
        if (a == b) return d[u] - d[v] == w;
        T t = d[u] - d[v] - w;
        if (sz[a] < sz[b])
        {
            swap(a, b);
            t = T(0) - t;
        }
        fa[b] = a;
        d[b] = t;
        sz[a] += sz[b];
        return true;
    }

    optional<T> diff(int u, int v)
    {
        int a = find(u), b = find(v);
        if (a != b) return nullopt;
        return d[u] - d[v];
    }

    int size(int u) { return sz[find(u)]; }
};
```

模板题使用：Library Checker Unionfind with Potential

题意：未知序列 a，1≤N,Q≤200000，模 998244353。0 u v x 尝试加入 a[u]−a[v]=x；与此前已接受信息一致输出 1，否则输出 0 并拒绝，不把矛盾边加入状态。1 u v 查询 a[u]−a[v] 的规范模值，未连通输出 −1；点号 0-based，0≤x<mod。重复一致、自约束及零差值均有效。先设置模整数的模数再构造结构，存活期间不改变它；复用已有加减与相等比较，不需要乘法逆元。

所需模板：PotentialDSU（第 7 页，2.1.3 节）、mint（第 121 页，A.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/unionfind_with_potential

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    using Z = mint<>;
    Z::set_mod(998244353);
    PotentialDSU<Z> d(n);
    while (q--)
    {
        int t, u, v;
        cin >> t >> u >> v;
        if (t == 0)
        {
            int x;
            cin >> x;
            cout << d.merge(u, v, Z(x)) << '\n';
        }
        else
        {
            auto x = d.diff(u, v);
            cout << (x ? x->v : -1) << '\n';
        }
    }
}
```

2.1.4 时间线段树分治：离线动态连通性

问题描述：离线动态图连通性：已知每条边在哪段时间区间内存在（加边、删边已配成区间），对每个时刻 t 回调一次，回调中可查询此刻的连通性和连通块大小。

核心思路：线段树分治：在时间轴 $[0, q)$ 上建线段树，每条边的存在区间拆成 $O(\log q)$ 个节点挂入 `edges[p]`。DFS 遍历线段树，进入节点时把挂在上面的边并入可撤销并查集 `RollbackDSU`，到达叶子 t 时并查集恰好包含此刻存在的全部边，调用 `visit`；离开节点时回滚到进入前的快照。

代码依赖：RollbackDSU（第 5 页，2.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：TimeConnectivity(n,q)，点号 0..n（1-based 图不使用 0），时间 0..q-1。add(l,r,u,v) 登记边在半开区间 [l,r) 有效，允许空区间、重叠区间、重边和自环。run(visit) 按时间递增调用 visit(t,const RollbackDSU &d)，可用 find 判断连通、siz[find(u)] 取分量大小；只在回调内读取，不保留引用，不在遍历中改图。每次 run 重新建并查集，可重复调用，也可在两次 run 间加区间。q=0 不调用回调。n>=0 且 n<INT_MAX，0<=q<=INT_MAX/4，并须满足内存限制。K 个非空区间：存储 O(n+q+K log(q+1))；建区间 O(K log(q+1))，遍历 O(n+q+K log(q+1) log(n+2))，另加回调代价。依赖 RollbackDSU，不做路径压缩；DFS 保持递归。

```
struct TimeConnectivity
{
```

```
    int n, q;
    vector<vector<pair<int, int>>> edges;

    TimeConnectivity(int n, int q) : n(n), q(q), edges(4 * size_t(q) + 1)
    {
        assert(n >= 0 && n < INT_MAX && q >= 0 && q <= INT_MAX / 4);
    }

    // Edge (u,v) exists at times l <= t < r. Empty intervals are allowed.
    void add(int l, int r, int u, int v)
    {
        assert(0 <= l && l <= r && r <= q);
        assert(0 <= u && u <= n && 0 <= v && v <= n);
        if (l < r)
            insert(1, 0, q, l, r, {u, v});
    }

    template <class F> void run(F visit) const
    {
        RollbackDSU d(n);
        if (q > 0)
            dfs(1, 0, q, d, visit);
    }
```

时间区间插入与回滚递归（接上页同一结构体）：

```
private:
    void insert(int p, int l, int r, int a, int b, pair<int, int> e)
    {
        if (a <= l && r <= b)
        {
            edges[p].push_back(e);
            return;
        }
        int m = l + (r - l) / 2;
        if (a < m)
            insert(p * 2, l, m, a, b, e);
        if (m < b)
            insert(p * 2 + 1, m, r, a, b, e);
    }

    template <class F>
    void dfs(int p, int l, int r, RollbackDSU &d, F &visit) const
    {
        int saved = d.snapshot();
        for (auto [u, v] : edges[p])
            d.merge(u, v);
        if (r - l == 1)
            visit(l, static_cast<const RollbackDSU &>(d));
        else
        {
            int m = l + (r - l) / 2;
            dfs(p * 2, l, m, d, visit);
            dfs(p * 2 + 1, m, r, d, visit);
        }
        d.rollback(saved);
    }
};
```

模板题使用：LOJ121 动态图连通性

题意：初始空的无向简单图，点号 1..n；0 加不存在的边、1 删存在的边、2 查连通性，输出 Y/N。n≤5000，q≤500000；端点统一排序，加删配成半开生存区间，剩余边持续到 q。

所需模板：RollbackDSU（第 5 页，2.1.2 节）、TimeConnectivity（第 9 页，2.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/121

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity tree(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> query(q, {-1, -1});
    for (int t = 0; t < q; t++)
    {
        int op, u, v;
        cin >> op >> u >> v;
        if (u > v) swap(u, v);
        pair<int, int> edge{u, v};
        if (op == 0)
            start[edge] = t;
        else if (op == 1)
        {
            auto it = start.find(edge);
            tree.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            query[t] = edge;
    }
    for (auto [edge, l] : start)
        tree.add(l, q, edge.first, edge.second);
    tree.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = query[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? 'Y' : 'N') << '\n';
    });
}
```

2.1.5 并查集模板题的接口范围

P3367 输入点号为 1-based，统一减一后调用第 3 页的 `dsu`（merge、same）。

第三章　堆

3.1　配对堆

3.1.1　PBDS 可修改、可合并优先队列

功能：GNU pb_ds 的配对堆优先队列（默认大根堆）：除普通的入队、取堆顶、出队外，`push` 返回元素句柄，可通过句柄修改或删除任意元素，并能把一个堆整体合并进另一个堆。

核心思路：配对堆是一棵多叉堆序树：插入和合并只把两个根中较劣的挂到较优者下面，均为常数时间；弹出堆顶时把根的所有孩子两两配对合并、再从右往左依次合并成新树，摊还对数。句柄 `point_iterator` 直接指向树节点，修改或删除时把该节点从树中摘下，调整后再与根合并。

接口契约：pheap<T,C> 默认比较器为 less<T>，固定 pairing_heap_tag，形成大根堆；小根堆用 greater<T>。

包含 ext/pb_ds/priority_queue.hpp。push(x) 返回 point_iterator 句柄；modify(h,x) 改值，erase(h) 删除指定元素。top/pop 要求非空，size/empty/clear 沿用容器接口。a.join(b) 把 b 全部移入 a，b 变空；原 b 元素句柄仍指向同一元素，此后应交给 a。a.swap(b) 后句柄随元素交换归属。split(pred,b) 将满足谓词的元素移入 b，本模板约定 b 原先为空。join/split 两个堆须不同，比较规则一致。删除元素、pop 对应堆顶、clear 或销毁后，相应句柄失效；不要把原堆句柄用于深复制的副本。top、push、join 为 O(1)；pop 摊还 O(log n)，modify/erase 的单次最坏可达 O(n)，不当作最坏常数。split/复制/clear 可能扫描整个堆。适合需要修改已入堆元素或合并的任务；只要普通入队出队时可用 std::priority_queue。复杂度按比较、复制标量或编号对为常数计，T 须满足容器的复制要求。

```
template <class T, class C = less<T>>
using pheap = __gnu_pbds::priority_queue<T, C, __gnu_pbds::pairing_heap_tag>;
```

```
pheap<int> a, b;
auto h = a.push(7);
a.modify(h, 2);
b.join(a);
b.erase(h);
```

模板题使用：Library Checker Double-Ended Priority Queue

题意：多重集合支持插入和删除一个最小/最大值；0≤n≤500000、1≤q≤500000，值在 $[-10^9,10^9]$，删除时保证非空。用两个大根配对堆分别保存 (-值,id)/(值,id)，同次插入保存两份句柄；删除时先 erase 另一堆的句柄，再 pop 当前堆。id 每次新插入都递增，重复值也可区分。删除后两份句柄失效，禁止再次使用。本题取负在 int 范围内；全 int64 值不能直接照搬取负。

所需模板：pheap（第 13 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/double_ended_priority_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using H = pheap<pair<int, int>>;
    array<H, 2> q;
    vector<array<H::point_iterator, 2>> h;
    h.reserve(n + m);
    auto add = [&](int x)
    {
        int id = h.size();
        h.push_back({q[0].push({-x, id}), q[1].push({x, id})});
    };
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        add(x);
    }
    while (m--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int x;
            cin >> x;
            add(x);
        }
        else
        {
            int k = t - 1;
            auto [x, id] = q[k].top();
            cout << (k ? x : -x) << '\n';
            q[k ^ 1].erase(h[id][k ^ 1]);
            q[k].pop();
        }
    }
}
```

3.2 左偏树

3.2.1 左偏树可并小根堆

功能：维护多个小根堆，支持新建单元素堆、合并两个堆、删除堆顶，堆以根节点编号表示。

核心思路：左偏树：节点的 `dist` 为到最近空孩子的距离，始终保持左孩子 `dist` 不小于右孩子，故右链长度为对数级。`meld(x,y)` 取 (key, 编号) 较小者为根，把另一堆与它的右子树递归合并，回溯时若左 `dist` 更小就交换左右孩子，并令 `dist` = `dist`(右) + 1。`pop` 合并根的左右子树并把根重置为孤立点。

接口契约：默认构造为空节点池；singleton(key) 创建单点堆并返回节点编号，编号从 1 开始，0 表示空堆。meld(x,y) 原地合并两个不相交堆并返回新根，调用后旧根不能再当独立堆使用；pop(root) 删除非空堆的最小节点并返回新根，必须保存返回值。相同 key 按节点编号小者优先，t[root].key 为最小值，root 本身就是最小节点编号。key 支持完整 long long；仅比较而不做减法。dist[空]=0，叶子 dist=1，每点左子树 dist 不小于右子树；递归沿右链合并，meld/pop 为 O(log(n+1))，其中 n 为参与堆节点数。singleton 均摊 O(1)，节点池空间 O(累计创建数)，节点不回收，不保留版本；pop 后删除节点已脱离堆，不应继续作为根操作。只保证根操作，不提供任意点所属堆查询，P3377 驱动另用并查集和删除标记维护。不要改写内部 l/r/dist，也不要合并有交集的堆；assert(x!=y) 不能检测所有非法重叠。

```
struct LeftistHeap
{
    struct Node
    {
        long long key;
        int l, r, dist;
    };

    vector<Node> t{{0, 0, 0, 0}};

    int singleton(long long key)
    {
        t.push_back({key, 0, 0, 1});
        return int(t.size()) - 1;
    }

    int meld(int x, int y)
    {
        if (!x || !y) return x ? x : y;
        assert(x != y);
        if (pair(t[x].key, x) > pair(t[y].key, y)) swap(x, y);
        t[x].r = meld(t[x].r, y);
        if (t[t[x].l].dist < t[t[x].r].dist) swap(t[x].l, t[x].r);
        t[x].dist = t[t[x].r].dist + 1;
        return x;
    }

    int pop(int x)
    {
        assert(x > 0);
        int root = meld(t[x].l, t[x].r);
        t[x].l = t[x].r = 0;
        t[x].dist = 1;
        return root;
    }
};
```

模板题使用：Luogu P3377

题意：合并指定元素所在的堆，或弹出其堆最小值；已删除元素的操作无效。同值先删先输入者。并查集代表不一定是堆根，root 单独保存；节点编号比输入数组下标大 1。

所需模板：LeftistHeap（第 15 页，3.2.1 节）、dsu（第 3 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3377

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LeftistHeap heap;
    dsu sets(n);
    vector<int> root(n);
    vector<bool> removed(n);
    for (int i = 0; i < n; i++)
    {
        long long key;
        cin >> key;
        root[i] = heap.singleton(key);
    }
    while (m--)
    {
        int op, x;
        cin >> op >> x;
        x--;
        if (op == 1)
        {
            int y;
            cin >> y;
            y--;
            if (removed[x] || removed[y]) continue;
            int a = sets.find(x), b = sets.find(y);
            if (a == b) continue;
            root[a] = heap.meld(root[a], root[b]);
            sets.merge(a, b);
        }
        else if (removed[x])
            cout << -1 << '\n';
        else
        {
            int a = sets.find(x), id = root[a];
            cout << heap.t[id].key << '\n';
            removed[id - 1] = true;
            root[a] = heap.pop(id);
        }
    }
}
```

第四章　ST 表

4.1　位掩码静态区间最值与位置

功能：静态数组上的区间最值查询：给定 $[l,r)$，返回区间内最优（默认最小）元素的位置，并列取最左。可传入比较器改为最大值或其他严格弱序。

核心思路：按长度 b（约 $\log_2 n$）分块。块内从左到右维护单调栈，并用位掩码表示：`mask[i]` 的第 j 位为 1 表示块内位置 j 在“块首到 i”的单调栈中，即它之后到 i 没有严格更优的元素。于是块内区间 $[l,r)$ 的答案是 `mask[r-1]` 中不低于 l 的最低位。每块的最优位置组成数组，再建 ST 表。跨块询问拆成左块后缀、右块前缀和中间整块三部分，用 `better` 取优（相等取下标小者）。

思路中的情况	代码中的处理	行
块内单调栈	新元素严格更优时弹掉掩码最高位（栈顶）	23
块最优值	块末掩码的最低位即整块最左最优位置	31
块间 ST 表	倍增合并相邻两段的最优位置	40
块内询问	掩码右移掉 l 之前的位后取最低位	54
跨块询问	左块后缀与右块前缀先比较，中间整块查 ST 表	62

接口契约：StaticRMQ<T,Cmp>(a,cmp) 复制数组；默认 T=long long、Cmp=less<T> 求最小，greater<T> 求最大，也可传有状态比较器。query(l,r) 返回 0-based 半开区间 [l,r) 内最优元素的原编号，并列总取最左；值读 rmq.a[编号]。要求 0≤l<r≤n；空数组可构造但不可查询；重新赋值新对象重建，不支持修改。比较器必须是稳定的严格弱序、const 可调用，元素类型可复制；浮点数据不能含 NaN。

单调栈位掩码处理块内区间，块间用 ST 表；块长为 n 的二进制位数（空数组取 1），n≤INT_MAX/2 且内存可分配，32 位掩码足够。在复制/比较/位操作 O(1) 的模型下，预处理 O(n)、查询 O(1)、空间 O(n)。所有成员约定只读，better/small 是内部辅助；small 仅供非空同块区间。只比较元素，不做元素算术，signed64 极值合法。

```cpp
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ
{
    int n, b;
    vector<T> a;
    vector<uint32_t> mask;
    vector<vector<int>> st;
    Cmp cmp;

    StaticRMQ(const vector<T> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        b = max(1, (int)bit_width((uint32_t)n));
        mask.resize(n);
        if (!n) return;
        st.push_back({});
        uint32_t s = 0;
        for (int i = 0; i < n; i++)
        {
            int start = i - i % b;
            if (i % b == 0) s = 0;
            while (s && cmp(a[i], a[start + bit_width(s) - 1]))
            {
                s ^= uint32_t(1) << (bit_width(s) - 1);
            }
            s |= uint32_t(1) << (i % b);
            mask[i] = s;
            if (i % b == b - 1 || i == n - 1)
            {
                st[0].push_back(start + countr_zero(s));
            }
        }
```

块间表构建、最优位置合并与查询（接上页同一结构体）：

```
        int m = st[0].size();
        for (int k = 1; (1 << k) <= m; k++)
        {
            st.push_back(vector<int>(m - (1 << k) + 1));
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    int better(int x, int y) const
    {
        if (cmp(a[x], a[y])) return x;
        if (cmp(a[y], a[x])) return y;
        return min(x, y);
    }

    int small(int l, int r) const
    {
        return l + countr_zero(mask[r - 1] >> (l % b));
    }

    int query(int l, int r) const
    {
        assert(0 <= l && l < r && r <= n);
        int x = l / b, y = (r - 1) / b;
        if (x == y) return small(l, r);
        int ans = better(small(l, (x + 1) * b), small(y * b, r));
        x++;
        if (x < y)
        {
            int k = bit_width((uint32_t)(y - x)) - 1;
            ans = better(ans, better(st[k][x], st[k][y - (1 << k)]));
        }
        return ans;
    }
};
```

模板题使用：Library Checker Static RMQ

题意：静态数组上查询 0-based 半开区间 [l,r) 最小值；n,q≤500000，值为 0..10^9。模板返回最左最小值位置，通过 rmq.a[位置] 输出值。

所需模板：StaticRMQ（第 17 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/staticrmq

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l, r)] << '\n';
    }
}
```

第五章　树状数组

5.1　四矩二维树状数组：矩形加与矩形和

功能：维护 $n \times m$ 的整数矩阵（初值全 0），支持给一个子矩形整体加 v，以及查询一个子矩形的元素和。

核心思路：矩形加在二维差分数组 D 上只改四个角（`corner`）。前缀和 $S(x,y)=\sum_{a<x,b<y} D(a,b)(x-a)(y-b) = xy\sum D - y\sum aD - x\sum bD + \sum abD$，所以用一棵二维树状数组同时维护 D, aD, bD, abD 四个量（`bit[i][j]` 的四个分量），`prefix` 求出四个和后按上式组合，矩形和再由四个前缀容斥。落在右边界或上边界的角不影响网格，直接跳过。

接口契约：RectangleFenwick<T>(n,m)，T 默认 long long，0≤n,m<2^30，初始化全 0；稠密空间与构造 O((n+1)(m+1))，需先确认可分配。add(l,d,r,u,v) 给 0-based 半开矩形 [l,r)×[d,u) 加 v；sum(l,d,r,u) 查询同一区间，prefix(x,y) 查询 [0,x)×[0,y)。端点须在 [0,n]×[0,m]，允许空矩形及零维矩阵。差分四角维护 D、xD、yD、xyD 四个矩，前缀和为 xyS0-ySx-xSy+Sxy；每次操作 O(log n log m)，实际存储每格 4 个 T。单点操作令右上界各加 1。T 支持零值、加减、取负和乘 int；包括-v、各坐标乘积及容斥在内的中间值均须可表示，仅最终答案可表示不够。内部 bit 与 corner 不供直接修改。重新赋值 RectangleFenwick(n,m) 清空；不支持稀疏大坐标自动压缩，因为区间加仍取决于真实格子数量。

```
template <class T = long long> struct RectangleFenwick
{
    int n, m;
    vector<vector<array<T, 4>>> bit;

    RectangleFenwick(int n, int m) : n(n), m(m)
    {
        assert(0 <= n && n < (1 << 30) && 0 <= m && m < (1 << 30));
        bit.assign(n + 1, vector<array<T, 4>>(m + 1));
    }

    // Difference impulse at (x,y); boundary impulses do not affect the grid.
    void corner(int x, int y, T v)
    {
        if (x == n || y == m) return;
        array<T, 4> a{v, v * x, v * y, v * x * y};
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j)
                for (int k = 0; k < 4; k++) bit[i][j][k] += a[k];
    }

    // Add to [l,r) x [d,u), 0-based. Empty rectangles are allowed.
    void add(int l, int d, int r, int u, T v)
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return;
        corner(l, d, v);
        corner(r, d, -v);
        corner(l, u, -v);
        corner(r, u, v);
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    T prefix(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= m);
        array<T, 4> a{};
        for (int i = x; i; i -= i & -i)
            for (int j = y; j; j -= j & -j)
                for (int k = 0; k < 4; k++) a[k] += bit[i][j][k];
        return a[0] * x * y - a[1] * y - a[2] * x + a[3];
    }

    T sum(int l, int d, int r, int u) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Luogu P4514 / 上帝造题的七分钟

题意：输入 X n m 初始化零矩阵，n,m≤2048；读到 EOF，L a b c d delta 给 1-based 闭矩形加 delta，k a b c d 查询区域和（小写 k）。至多 200000 操作，|delta|≤500；转换为 [a-1,c)×[b-1,d)。最终答案在 32 位内，中间值未保证，使用 long long。

所需模板：RectangleFenwick（第 21 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4514

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    char op;
    int n, m;
    std::cin >> op >> n >> m;
    RectangleFenwick tree(n, m);
    int a, b, c, d;
    while (std::cin >> op >> a >> b >> c >> d)
    {
        if (op == 'L')
        {
            long long v;
            std::cin >> v;
            tree.add(a - 1, b - 1, c, d, v);
        }
        else
            std::cout << tree.sum(a - 1, b - 1, c, d) << '\n';
    }
}
```

5.2 树状数组第 k 小与组合依赖

功能：维护长度为 n 的数组，支持单点加、前缀和与区间和；元素表示频数（均非负）时还能求“前缀和首次达到 k 的位置”，即可重集合的第 k 小。

核心思路：`a[x]` 存原数组在 $(x-\text{lowbit}(x), x]$ 上的和。前缀和沿 $x \leftarrow x-\text{lowbit}(x)$ 累加，单点修改沿 $x \leftarrow x+\text{lowbit}(x)$ 更新所有覆盖它的节点，各 $O(\log n)$。第 k 小从高位到低位倍增：若跳过下一段后累计仍小于 k 就跳过去。

接口契约：保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 23 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

相关应用：离线二维扫描线应用，不代表整页覆盖

5.3 离线静态矩形权值和

问题描述：给定平面上若干带权点（不修改），多次询问左闭右开矩形 $[l,r)\times[d,u)$ 内点的权值和，所有询问一次读入后统一回答。

核心思路：把询问拆成两条竖直扫描线：答案 $= F(r) - F(l)$，其中 $F(X)$ 是横坐标小于 X、纵坐标在 $[d, u)$ 内的点权和。点按横坐标排序，事件按扫描位置排序；处理位置为 X 的事件前，把所有横坐标严格小于 X 的点加入按纵坐标离散化的树状数组，再用两次前缀和求纵坐标落在 $[d, u)$ 的权值和，乘上事件的符号 ± 1 累加到对应询问。

代码依赖：Fenwick（第 23 页，5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：offline_rectangle_sum(points,queries)：points 每项为 {x,y,w}，queries 每项为 {l,d,r,u}，返回按询问原顺序排列的 [l,r)×[d,u) 内权值和。坐标、端点与权值均为 long long；要求 l<=r、d<=u，允许空矩形、空点集、空询问、重合点和负权。传入点按值复制，两个输入均不改变，多次调用互不影响。坐标仅比较，不用端点 ±1，支持 LLONG_MIN/MAX；半开矩形无法包含位于 LLONG_MAX 的上边界坐标。

一次读完所有询问，按横坐标扫描，在纵轴离散化树状数组上加点。同横坐标事件只包含 x< 事件横坐标的点，保证左右开闭正确。时间 O((N+Q)log(N+Q+1))，空间 O(N+Q)，N,Q 均小于 2^30 且内存可分配。要求所有权值绝对值之和 <=LLONG_MAX，保证树状数组、容斥与符号乘法不溢出。依赖 Fenwick；若需逐条在线回答，改用持久化前缀版本。

```
vector<long long> offline_rectangle_sum(vector<array<long long, 3>> points,
                                        const vector<array<long long, 4>> &queries)
{
    using ll = long long;
    assert(points.size() < (1U << 30) && queries.size() < (1U << 30));
    vector<ll> ys;
    for (auto [x, y, w] : points)
        ys.push_back(y);
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](ll y)
    {
        return int(lower_bound(ys.begin(), ys.end(), y) - ys.begin());
    };
    struct Event
    {
        ll x;
        int d, u, id, sign;
    };
    vector<Event> events;
    events.reserve(2 * queries.size());
    for (int i = 0; i < (int)queries.size(); i++)
    {
        auto [l, d, r, u] = queries[i];
        assert(l <= r && d <= u);
        if (l == r || d == u)
            continue;
        events.push_back({l, rank(d), rank(u), i, -1});
        events.push_back({r, rank(d), rank(u), i, 1});
    }
    sort(points.begin(), points.end());
    sort(events.begin(), events.end(), [](const Event &a, const Event &b)
    {
        return a.x < b.x;
    });
    Fenwick<ll> bit(ys.size());
    vector<ll> answer(queries.size());
    int p = 0;
    for (auto e : events)
    {
```

```
        while (p < (int)points.size() && points[p][0] < e.x)
        {
            auto [x, y, w] = points[p];
            bit.add(rank(y) + 1, w);
            p++;
        }
        ll sum = bit.sum(e.u) - bit.sum(e.d);
        answer[e.id] += e.sign * sum;
    }
    return answer;
}
```

模板题使用：Library Checker Rectangle Sum（离线）

题意：N,Q<=200000，点坐标与权值在 [0,1e9]；读取全部询问后用扫描线返回半开矩形权值和。此例可离线处理，节省持久化节点的内存；需要在线查询时使用持久化版本（用法 432）。Library Checker C++20 线上 AC 410016，258ms、29.98MiB。

所需模板：Fenwick（第 23 页，5.2 节）、offline_rectangle_sum（第 24 页，5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<long long, 3>> points(n);
    vector<array<long long, 4>> queries(q);
    for (auto &p : points)
        cin >> p[0] >> p[1] >> p[2];
    for (auto &r : queries)
        cin >> r[0] >> r[1] >> r[2] >> r[3];
    for (auto answer : offline_rectangle_sum(points, queries))
        cout << answer << '\n';
}
```

第六章　线段树

6.1　线段树基础

6.1.1　通用懒标记线段树与边界搜索

功能：对由幺半群 $(S, \mathtt{op}, \mathtt{e})$ 元素组成的数组，支持区间作用一个映射 $f \in F$、区间按序合并、单点读写，以及从某端点出发二分求满足谓词的最远边界。具体问题只需给出 S、F 和五个函数。

核心思路：递归线段树，`d[p]` 是节点区间的合并结果（已包含本节点标记的作用），`tag[p]` 是尚未下传给孩子的映射。区间作用在完全覆盖的节点上调用 `act`：用 `mapping` 更新 `d`，用 `composition` 把新映射复合到旧标记外层；进入孩子前 `push`，回溯时 `pull`。`max_right` 从左往右累积 s，整块合并后仍满足谓词就整块跳过，否则下传并进入该块，先左后右找到第一个使谓词变假的叶子；`min_left` 方向相反、合并顺序为 $\mathtt{op}(\mathtt{d[p]}, s)$。

接口契约：模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
133     template <class G> int min_left(int r, G ok)
134     {
135         assert(0 <= r && r <= n && ok(e()));
136         S s = e();
137         return r == 0 ? 0 : left(1, 0, n, r, s, ok);
138     }
139 
140     template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
141     {
142         if (b <= l) return 0;
143         if (r <= b && ok(op(d[p], s)))
144         {
145             s = op(d[p], s);
146             return 0;
147         }
148         if (r - l == 1) return r;
149         push(p);
150         int m = (l + r) / 2;
151         int x = left(p * 2 + 1, m, r, b, s, ok);
152         return x != 0 ? x : left(p * 2, l, m, b, s, ok);
153     }
154 };
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 28 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

6.1.2 通用线段树与左右边界搜索

功能：对由幺半群 $(S, \mathtt{op}, \mathtt{e})$ 元素组成的数组，支持单点赋值、区间按从左到右的顺序合并，以及从一个端点出发求满足单调谓词的最远另一端点。

核心思路：非递归线段树：长度补到 2 的幂 `sz`，叶子在 `d[sz+i]`，内部节点 p 合并 $2p, 2p+1$。`prod` 从两端向上爬，左侧结果 `a` 在右边追加、右侧结果 `b` 在左边追加，保证不交换律也正确。`max_right` 从 l 起每次跳到当前位置所在的最大完整块，若并入后谓词变假，就在该块内向下二分（能并入左孩子则并入并转向右孩子）；`min_left` 对称地从右往左，并始终以 $\mathtt{op}(\mathtt{d[r]}, a)$ 的顺序合并。

接口契约：segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 34 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

相关应用：线段树维护单调栈的双向极大值贡献；区间加接雨水应用，不将普通单调栈直接视为覆盖。

6.1.3 线段树维护单调栈：区间加与接雨水

功能：维护高度数组，支持区间加；查询整个数组的接雨水体积 $\sum_i(\min(\text{前缀最大}, \text{后缀最大}) - a_i)$，以及从给定初始高度 h 出发向左或向右扫描一段时 $\sum(\text{当前最大值} - a_i)$。

核心思路：“线段树维护单调栈”：节点 `fill[0]`、`fill[1]` 缓存本区间自左（自右）扫描时 $\sum(\text{运行最大值} - a_i)$。`extra(p,h,dir)` 求以初值 h 扫过节点的同一量：h 不小于整段最大值时为 $h \cdot$ `len` $-$ `sum`；不小于先扫孩子的最大值时先扫孩子全为 h，递归另一孩子；否则递归先扫孩子，另一孩子的贡献只取决于先扫孩子的最大值，等于 `fill`$(p)-$`fill`(先扫孩子)。故每次 `extra` 只走一条链。由 $\min(P,S) = P+S-\max$, `water` 为 `fill[0]`$+$`fill[1]`$-(n \cdot \max - \sum a)$。

接口契约：MonotoneStackSeg(a) 要求非空 long long 数组；add(l,r,delta) 对 0-based 半开区间加值，空区间不变。water() 返回整个数组的接雨水体积。scan(l,r,height,reverse=false) 按左到右（reverse=true 则右到左）扫描，将 height 与当前元素取 max 并累加 height-a[i]，返回 (累加值, 最后的 height)；空区间返回 (0,height)。支持负高度与负增量，无固定零边界。节点缓存双向前缀极大值减原值的和；区间平移不改变两者差。建树 O(n)，add/scan 最坏 O($\log^2$(n+1))，water 为 O(1)，空间 O(n)；递归深度 O(log n)。4n 须可由 int 表示；高度、和、懒标记、外部 height 与长度的乘积以及所有中间运算均须在 long long 范围内。t 与 extra/pull/push 及带节点编号的函数属内部实现。

```
struct MonotoneStackSeg
{
    using ll = long long;

    struct Node
    {
        int len = 0;
        ll mx = 0, sum = 0, lazy = 0;
        array<ll, 2> fill{};
    };

    int n;
    vector<Node> t;

    MonotoneStackSeg(const vector<ll> &a) : n(a.size()), t(4 * n)
    {
        assert(n > 0);
        build(1, 0, n, a);
    }

    void apply(int p, ll delta)
    {
        t[p].mx += delta;
        t[p].sum += delta * t[p].len;
        t[p].lazy += delta;
    }

    void push(int p)
    {
        if (!t[p].lazy) return;
        apply(p * 2, t[p].lazy);
        apply(p * 2 + 1, t[p].lazy);
        t[p].lazy = 0;
    }
```

单路扫描、合并与建树（接上页同一结构体）：

```
ll extra(int p, ll height, int dir)
{
    if (height >= t[p].mx) return height * t[p].len - t[p].sum;
    if (t[p].len == 1) return 0;
    push(p);
    int first = p * 2 + dir, second = p * 2 + 1 - dir;
    if (height >= t[first].mx)
        return height * t[first].len - t[first].sum + extra(second, height, dir);
    return extra(first, height, dir) + (t[p].fill[dir] - t[first].fill[dir]);
}

void pull(int p)
{
    int l = p * 2, r = l + 1;
    t[p].len = t[l].len + t[r].len;
    t[p].mx = max(t[l].mx, t[r].mx);
    t[p].sum = t[l].sum + t[r].sum;
    t[p].fill[0] = t[l].fill[0] + extra(r, t[l].mx, 0);
    t[p].fill[1] = t[r].fill[1] + extra(l, t[r].mx, 1);
}

void build(int p, int l, int r, const vector<ll> &a)
{
    if (r - l == 1)
    {
        t[p].len = 1;
        t[p].mx = a[l];
        t[p].sum = a[l];
        return;
    }
    int m = (l + r) / 2;
    build(p * 2, l, m, a);
    build(p * 2 + 1, m, r, a);
    pull(p);
}
```

区间加、双向查询与雨水体积（接上页同一结构体）：

```
    void add(int p, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr) return apply(p, delta);
        push(p);
        int m = (l + r) / 2;
        if (ql < m) add(p * 2, l, m, ql, qr, delta);
        if (m < qr) add(p * 2 + 1, m, r, ql, qr, delta);
        pull(p);
    }

    void add(int l, int r, ll delta)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) add(1, 0, n, l, r, delta);
    }

    ll scan(int p, int l, int r, int ql, int qr, ll &height, int dir)
    {
        if (ql <= l && r <= qr)
        {
            ll answer = extra(p, height, dir);
            height = max(height, t[p].mx);
            return answer;
        }
        push(p);
        int m = (l + r) / 2;
        ll answer = 0;
        if (dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        if (ql < m) answer += scan(p * 2, l, m, ql, qr, height, dir);
        if (!dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        return answer;
    }

    // Return (sum of running maximum - a[i], final running maximum).
    pair<ll, ll> scan(int l, int r, ll height, bool reverse = false)
    {
        assert(0 <= l && l <= r && r <= n);
        ll answer = 0;
        if (l < r) answer = scan(1, 0, n, l, r, height, reverse);
        return {answer, height};
    }

    ll water() const
    {
        ll gap = t[1].mx * n - t[1].sum;
        return t[1].fill[0] - (gap - t[1].fill[1]);
    }
};
```

6.1.4 ACL 的组合约定

懒标记 composition(f,g) 表示先作用 g、再作用 f；SCC 分量方向与 2-SAT 的真假编码须配套，不能分别替换。Fenwick 的取模区间和不适用于 kth。

6.1.5 线段树维护单调栈：双向扫描与接雨水

`MonotoneStackSeg` 每段存 F_0, F_1：从左、从右扫描的运行最大值减原值之和。外部高度 $x \geq$ 段最大值时贡献 $x \cdot len - sum$；否则按扫描顺序孩子 A, B：$x \geq \max A$ 递归 B，否则递归 A 并加 $F_{parent} - F_A$。单段扫描 $O(\log n)$，区间修改及任意子区间扫描最坏 $O(\log^2(n+1))$，建树 $O(n)$。雨水体积（M 为全局最大值）根上 $O(1)$：

$$\sum_i \big(\min(L_i, R_i) - a_i\big) = F_0 + F_1 - \big(nM - \sum_i a_i\big).$$

负高度可用；所有中间量须在 long long 内。

6.2 线段树合并 & 分裂

6.2.1 线段树合并与分裂

功能：维护多个可重集合，元素是值域 $[0, n)$ 内的整数：支持增减某个值的个数、把一个集合并入另一个、按值域区间或按排名（最小或最大的 k 个）分出一个新集合，以及查询值域区间计数与第 k 小。

核心思路：每个集合是一棵动态开点的权值线段树，节点 `sum` 为该值域的元素个数，个数变为 0 的节点回收到 `free` 复用。合并时两树同步递归，只在双方都有节点处继续，把 q 的计数并入 p 后回收 q。分裂时同步构造“留下”与“取走”两棵树：完全落入目标区间的子树整体移交，部分相交的节点为取走部分新建一个节点。按排名分裂时先算出左子树应取走多少个，叶子处可以拆分重数。

思路中的情况	代码中的处理	行
合并	一方为空就直接用另一方，叶子处累加计数，回收 q	107
值域分裂：子树完全在区间内	整棵子树移给新集合	136
值域分裂：部分相交	递归两侧后，原节点空了就回收；取走部分非空才新建节点	149
排名分裂：分配个数	取最小 k 个时左侧取 $\min(k, \text{左计数})$，取最大时左侧只取超出右计数的部分	188
排名分裂：叶子	同一值的重数拆成留下与取走两份	179
第 k 小	左子树计数不少于 k 走左，否则减去后走右	231

接口契约：MergeSplitTree(n) 建空集合 0，或传 vector<long long> 建初始重数；n>0，值域 [0,n)。add(s,x,delta) 点增减，结果重数非负；sum(s,l,r) 查询半开值域；kth(s,k) 按重数返回第 k 小的 0-based 值，不存在返回-1。split(s,l,r) 移走区间并返回新集合号，空区间也创建集合；split_rank(s,k,reverse=false) 移走最小 k 个元素到新集合；reverse=true 移走最大 k 个，k 在 [0, 集合总重数] 内，同值的重数可拆开，k=0 也创建空集合。merge(dst,src) 消耗不同的 src，src 变空后可重用。所有集合总量分别须 ≤LLONG_MAX，节点/集合索引须适合 int。仅通过集合号操作，root/t/free 只读，其余递归函数内部使用，禁止浅拷贝根。初始化 O(n)，增减/分裂/查询 O(log(n+1))；单次合并非 O(log n)，遍历重叠节点并回收，总合并开销可摊到节点创建。完整对象值复制独立。

```
struct MergeSplitTree
{
    using ll = long long;
    struct Node
    {
        int l = 0, r = 0;
        ll sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0}, free;

    MergeSplitTree(int n) : n(n)
    {
        assert(n > 0);
    }

    MergeSplitTree(const vector<ll> &a) : MergeSplitTree((int)a.size())
    {
        root[0] = build(0, n, a);
    }

    int make()
    {
        if (free.empty())
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free.back();
        free.pop_back();
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    void release(int p)
    {
        t[p] = Node{};
        free.push_back(p);
    }

    void pull(int p)
    {
        assert(t[t[p].l].sum <= LLONG_MAX - t[t[p].r].sum);
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int build(int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(a[l] >= 0);
            if (!a[l]) return 0;
            int p = make();
            t[p].sum = a[l];
            return p;
        }
        int m = l + (r - l) / 2;
        int x = build(l, m, a);
        int y = build(m, r, a);
        if (!x && !y) return 0;
        int p = make();
        t[p].l = x;
        t[p].r = y;
        pull(p);
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int add_node(int p, int l, int r, int x, ll v)
    {
        if (!p) p = make();
        if (r - l == 1)
        {
            assert(v >= -t[p].sum);
            t[p].sum += v;
        }
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = add_node(t[p].l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = add_node(t[p].r, m, r, x, v);
                t[p].r = q;
            }
            pull(p);
        }
        if (t[p].sum) return p;
        release(p);
        return 0;
    }

    // Sets and coordinates are 0-based; resulting multiplicities are nonnegative.
    void add(int s, int x, ll v)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= x && x < n);
        assert(v <= 0 || t[root[s]].sum <= LLONG_MAX - v);
        if (!v) return;
        root[s] = add_node(root[s], 0, n, x, v);
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int meld(int p, int q, int l, int r)
    {
        if (!p || !q) return p ? p : q;
        if (r - l == 1)
            t[p].sum += t[q].sum;
        else
        {
            int m = l + (r - l) / 2;
            t[p].l = meld(t[p].l, t[q].l, l, m);
            t[p].r = meld(t[p].r, t[q].r, m, r);
            pull(p);
        }
        release(q);
        return p;
    }

    // Consume src; its set ID remains valid and becomes empty.
    void merge(int dst, int src)
    {
        assert(0 <= dst && dst < (int)root.size());
        assert(0 <= src && src < (int)root.size() && dst != src);
        assert(t[root[dst]].sum <= LLONG_MAX - t[root[src]].sum);
        root[dst] = meld(root[dst], root[src], 0, n);
        root[src] = 0;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    pair<int, int> cut(int p, int l, int r, int a, int b)
    {
        if (!p || b <= l || r <= a) return {p, 0};
        if (a <= l && r <= b) return {0, p};
        int m = l + (r - l) / 2;
        auto [x, u] = cut(t[p].l, l, m, a, b);
        auto [y, v] = cut(t[p].r, m, r, a, b);
        t[p].l = x;
        t[p].r = y;
        pull(p);
        if (!t[p].sum)
        {
            release(p);
            p = 0;
        }
        int q = 0;
        if (u || v)
        {
            q = make();
            t[q].l = u;
            t[q].r = v;
            pull(q);
        }
        return {p, q};
    }

    // Move [l,r) into a new set, even when the extracted set is empty.
    int split(int s, int l, int r)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        auto [p, q] = cut(root[s], 0, n, l, r);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }

    pair<int, int> cut_rank(int p, int l, int r, ll k, bool reverse)
    {
        if (!p || k == 0)
            return {p, 0};
        if (k == t[p].sum)
            return {0, p};
        int q = make();
        if (r - l == 1)
        {
            t[p].sum -= k;
            t[q].sum = k;
            return {p, q};
        }
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        ll right = t[t[p].r].sum;
        ll take_left = k < left ? k : left;
```

```
        if (reverse)
            take_left = k > right ? k - right : 0;
        auto [a, b] = cut_rank(t[p].l, l, m, take_left, reverse);
        auto [c, d] = cut_rank(t[p].r, m, r, k - take_left, reverse);
        t[p].l = a;
        t[p].r = c;
        t[q].l = b;
        t[q].r = d;
        pull(p);
        pull(q);
        return {p, q};
    }

    // Move the k smallest (or largest) occurrences into a new set.
    int split_rank(int s, ll k, bool reverse = false)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= k && k <= t[root[s]].sum);
        auto [p, q] = cut_rank(root[s], 0, n, k, reverse);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    ll query(int p, int l, int r, int a, int b) const
    {
        if (!p || b <= l || r <= a) return 0;
        if (a <= l && r <= b) return t[p].sum;
        int m = l + (r - l) / 2;
        return query(t[p].l, l, m, a, b) + query(t[p].r, m, r, a, b);
    }

    ll sum(int s, int l, int r) const
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        return query(root[s], 0, n, l, r);
    }

    int select(int p, int l, int r, ll k) const
    {
        if (r - l == 1) return l;
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        if (k <= left) return select(t[p].l, l, m, k);
        return select(t[p].r, m, r, k - left);
    }

    int kth(int s, ll k) const
    {
        assert(0 <= s && s < (int)root.size());
        if (k <= 0 || k > t[root[s]].sum) return -1;
        return select(root[s], 0, n, k);
    }
};
```

模板题使用：Luogu P5494 / 线段树分裂

题意：n,m≤200000，初始集合 1 含每种值 1..n 的重数 ai≤m。0 p x y 移走闭区间到新集合；1 p t 合并并清空 t，之后不再使用 t；2 p x q 增加 x 个值 q；3 p x y 查区间重数；4 p k 查第 k 小，不存在-1。集合和值均转 0-based，闭区间转 [x-1,y)，重数用 long long。

所需模板：MergeSplitTree（第 42 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplitTree seg(a);
    for (int i = 0; i < m; i++)
    {
        int op, p;
        cin >> op >> p;
        p--;
        if (op == 0 || op == 3)
        {
            int x, y;
            cin >> x >> y;
            if (op == 0)
                seg.split(p, x - 1, y);
            else
                cout << seg.sum(p, x - 1, y) << '\n';
        }
        else if (op == 1)
        {
            int t;
            cin >> t;
            seg.merge(p, t - 1);
        }
        else if (op == 2)
        {
            long long x;
            int q;
            cin >> x >> q;
            seg.add(p, q - 1, x);
        }
        else
        {
            long long k;
            cin >> k;
            int x = seg.kth(p, k);
            cout << (x == -1 ? -1 : x + 1) << '\n';
        }
    }
}
```

6.3 李超线段树

6.3.1 精确线段李超树：最大值编号

功能：离线给出所有可能查询的横坐标，在线插入若干条线段（仅在其横坐标范围内有效），查询某个横坐标处纵坐标最大的线段编号；也可取出该线段在此处的精确值。

核心思路：李超线段树建在排序去重的查询横坐标 `xs` 上。插入时先把线段的横坐标范围拆成 $O(\log)$ 个完整节点；在每个节点比较新旧线段在中点 `xs[m]` 的值，较优者留在节点，较劣者只可能在一侧更优：左端点处更优就递归左孩子，右端点处更优就递归右孩子。查询从根走到叶，取路径上各节点线段的最优者。线段在 x 处的值写成分数 $\frac{y_1(x_2-x)+y_2(x-x_1)}{x_2-x_1}$，用 `__int128` 交叉相乘比较，相等时取编号小者。

接口契约：构造传入非空的所有查询横坐标，排序去重后数量 M。add(x1,y1,x2,y2) 插入闭线段并返回从 1 开始的编号；可交换端点，竖直线段在该横坐标取较高端点。query(x) 返回交点最高的线段编号，同值取较早插入者；无交点返回 0，x 必须在预登记坐标内。value(id,x) 返回未约分的 (分子, 正分母)，要求 id>0 且 x 在线段内；不直接相除即可保留精度。横坐标绝对值 ≤10^9，端点纵坐标绝对值 ≤2×10^18，int128 交叉乘积在这些界内安全；不使用浮点或 epsilon。最小值问题可先把纵坐标取负，得到编号后再还原。整条直线用原 LiChao 更短；这里插入仅作用于端点范围。构造 O(M log M)，插入 O($\log^2$(M+1))，查询 O(log(M+1))，空间 O(M+ 线段数)；要求 M<INT_MAX/4，累计插入线段数 <INT_MAX。

```
struct SegmentLiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll x1, y1, x2, y2;
    };

    vector<ll> xs;
    vector<Line> lines{{0, 0, 0, 0}};
    vector<int> t;

    SegmentLiChao(vector<ll> x) : xs(move(x))
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty() && xs.front() >= -1000000000 && xs.back() <= 1000000000);
        t.resize(4 * xs.size());
    }

    pair<I, I> value(int id, ll x) const
    {
        auto [a, b, c, d] = lines[id];
        if (a == c) return {max(b, d), 1};
        return {I(b) * (c - x) + I(d) * (x - a), c - a};
    }
```

精确比较与线段插入（接上页同一结构体）：

```
    bool better(int u, int v, ll x) const
    {
        if (!u || !v) return u != 0;
        auto [a, b] = value(u, x);
        auto [c, d] = value(v, x);
        return a * d == c * b ? u < v : a * d > c * b;
    }

    void insert(int p, int l, int r, int ql, int qr, int id)
    {
        int m = (l + r) / 2;
        if (ql <= l && r <= qr)
        {
            if (better(id, t[p], xs[m])) swap(id, t[p]);
            if (!id || l == r) return;
            if (better(id, t[p], xs[l]))
                insert(p * 2, l, m, l, m, id);
            else if (better(id, t[p], xs[r]))
                insert(p * 2 + 1, m + 1, r, m + 1, r, id);
            return;
        }
        if (ql <= m) insert(p * 2, l, m, ql, qr, id);
        if (qr > m) insert(p * 2 + 1, m + 1, r, ql, qr, id);
    }

    int add(ll x1, ll y1, ll x2, ll y2)
    {
        assert(-1000000000 <= x1 && x1 <= 1000000000);
        assert(-1000000000 <= x2 && x2 <= 1000000000);
        assert(-2000000000000000000LL <= y1 && y1 <= 2000000000000000000LL);
        assert(-2000000000000000000LL <= y2 && y2 <= 2000000000000000000LL);
        if (x1 > x2)
        {
            swap(x1, x2);
            swap(y1, y2);
        }
        int id = lines.size();
        lines.push_back({x1, y1, x2, y2});
        int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
        int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin() - 1;
        if (l <= r) insert(1, 0, (int)xs.size() - 1, l, r, id);
        return id;
    }
```

查询最优线段编号（接上页同一结构体）：

```
    int query(ll x) const
    {
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(i < (int)xs.size() && xs[i] == x);
        int p = 1, l = 0, r = (int)xs.size() - 1, ans = 0;
        while (true)
        {
            if (better(t[p], ans, x)) ans = t[p];
            if (l == r) return ans;
            int m = (l + r) / 2;
            if (i <= m)
            {
                p *= 2;
                r = m;
            }
            else
            {
                p = p * 2 + 1;
                l = m + 1;
            }
        }
    }
};
```

模板题使用：Library Checker segment_add_get_min

题意：在线段有效区间内查询最小值。预读所有查询横坐标以离散化，再按原序处理；没有查询时直接退出。题目横坐标都是整数，因此半开区间 [l,r) 换成闭区间 [l,r-1]。端点值取负，把最小值转为最大值；无结果输出 INFINITY，返回分数在本题必为整数。原线性式的端点计算在题目范围内可用 long long。

所需模板：SegmentLiChao（第 51 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/segment_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long l, r, a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        if (o.type == 0)
            cin >> o.l >> o.r >> o.a >> o.b;
        else
        {
            cin >> o.l;
            xs.push_back(o.l);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (xs.empty()) return 0;
    SegmentLiChao t(xs);
    for (auto o : ops)
        if (o.type == 0)
            t.add(o.l, -o.a * o.l - o.b, o.r - 1, -o.a * (o.r - 1) - o.b);
        else
        {
            int id = t.query(o.l);
            if (!id)
                cout << "INFINITY\n";
            else
            {
                auto [num, den] = t.value(id, o.l);
                cout << (long long)(-num / den) << '\n';
            }
        }
}
```

6.3.2 离散李超树与最优直线编号

功能：离线给出所有查询横坐标，在线插入整条直线 $y = kx + b$（各带调用者指定的编号），查询某个横坐标处的最小函数值及取到它的直线编号（同值取编号小者）。

核心思路：李超线段树建在排序去重的横坐标 `xs` 上，节点按需动态创建。每个节点保存一条在其区间中点 `xs[m]` 处最优的直线。插入时若新线在中点更优就与节点的线交换；两条直线至多相交一次，落败的那条只可能在一侧端点处更优，于是只递归进入那一侧（两侧都不优则丢弃）。查询沿根到对应叶子的路径，取路径上所有直线在 x 处的最优者。比较用 `__int128` 计算 $kx + b$，以 (值, 编号) 字典序决胜。

接口契约：构造传入所有可能查询的横坐标，add({k,b,id}) 插入直线，query(x) 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 int128，k、b、x 为有符号 64 位且中间值可表示。单次 O(log n)。

```
struct LiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll k = 0, b = 0;
        int id = -1;

        I value(ll x) const { return I(k) * x + b; }
    };

    struct Node
    {
        Line line;
        int left = -1, right = -1;
    };

    vector<ll> xs;
    vector<Node> a{Node{}};

    LiChao(vector<ll> x) : xs(x)
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
    }

    bool better(Line u, Line v, ll x) const
    {
        return v.id == -1 ||
               (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
    }

    void add(Line line)
    {
        assert(line.id >= 0);
        add(0, 0, (int)xs.size() - 1, line);
    }
```

插入与查询（接上页同一结构体）：

```
    void add(int p, int l, int r, Line line)
    {
        int m = (l + r) / 2;
        if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
        if (l == r || line.id == -1) return;
        bool left = better(line, a[p].line, xs[l]);
        bool right = better(line, a[p].line, xs[r]);
        if (!left && !right) return;
        int next = left ? a[p].left : a[p].right;
        if (next == -1)
        {
            next = (int)a.size();
            a.push_back(Node{});
            if (left)
                a[p].left = next;
            else
                a[p].right = next;
        }
        if (left)
            add(next, l, m, line);
        else
            add(next, m + 1, r, line);
    }

    pair<I, int> query(ll x) const
    {
        int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(idx < (int)xs.size() && xs[idx] == x);
        int p = 0, l = 0, r = (int)xs.size() - 1;
        Line best;
        while (p != -1)
        {
            if (better(a[p].line, best, x)) best = a[p].line;
            if (l == r) break;
            int m = (l + r) / 2;
            if (idx <= m)
            {
                p = a[p].left;
                r = m;
            }
            else
            {
                p = a[p].right;
                l = m + 1;
            }
        }
        assert(best.id != -1);
        return {best.value(x), best.id};
    }
};
```

模板题使用：Library Checker line_add_get_min

题意：插入整条直线并查询最小值。先收集查询横坐标，再按输入顺序处理；初始至少一条线，查询必有结果。编号由调用者递增指定，本题只输出值。题目系数与横坐标范围保证最终值可转为 long long；模板内部用 int128 比较。

所需模板：LiChao（第 56 页，6.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/line_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        cin >> o.a;
        if (o.type == 0)
            cin >> o.b;
        else
            xs.push_back(o.a);
    }
    if (xs.empty()) return 0;
    LiChao t(xs);
    int id = 0;
    for (auto o : ops)
        if (o.type == 0)
            t.add({o.a, o.b, id++});
        else
            cout << (long long)t.query(o.a).first << '\n';
}
```

6.4 区间最值操作 & 区间历史最值

6.4.1 Segment Tree Beats：区间截断、加法与和

功能：维护整数数组，支持区间对 x 取 min（$a_i \leftarrow \min(a_i, x)$）、区间取 max、区间加，以及区间求和。

核心思路：每个节点存和、最大值 mx、严格次大值 mx2、最大值个数 cmx，最小值一侧对称，另有加法标记。区间取 min 时：若 mx $\le x$ 无影响；若 mx2 $< x <$ mx，只有最大值们改变，和减去 $(\texttt{mx} - x) \cdot \texttt{cmx}$ 后打在本节点；否则继续递归（势能分析保证总复杂度）。取 max 对称。下传时先下传加法，再用父节点的 mx/mn 截断孩子，即父节点的最值本身充当截断标记。只有一两种值时最大值与最小值会互相影响，需要同步修改。

思路中的情况	代码中的处理	行
合并孩子	求最大、严格次大及个数，最小一侧对称	28
节点只需改最大值	和加 $(x - \texttt{mx}) \cdot \texttt{cmx}$；最小值或次小值等于原最大值时一起改	75
下传	先给两孩子加父标记，再按父 mx、mn 截断孩子	102
取 min 不影响该节点	mx $\le x$ 时直接返回（剪枝）	111
完全覆盖且 mx2 $< x$	直接打在本节点；否则下传并递归两侧	115

接口契约：SegmentBeats(a) 接受 0-based 数组，chmin/chmax/add(l,r,x)、sum(l,r) 均用半开区间。空数组/空区间允许，空和为 0，重新构造用于多测。维护最大/严格第二大/次数及最小对称项；阈值等于第二极值必须递归，先下传加法再按父极值截断。n≤(INT_MAX-4)/4 且内存可用；真实值、阈值、有限延迟极值、加法标记及先加后截断的中间值须严格在 ±2^60 内，返回和须可用 long long 表示。内部和与差值乘次数用 int128，避免尚未同步的孩子临时总和溢出；不能由最终值有界推断中间和安全。建树/空间 O(n)，sum/add 为 O(log n)，单次 chmin/chmax 最坏 O(n)，混合操作总摊还 $O((n+q)\log^2(n+1))$。sum 会下传，不是 const。内部辅助函数与 t 不得直接调用/改写；不维护历史最值。

```
struct SegmentBeats
{
    using ll = long long;
    static constexpr ll inf = 1LL << 60;

    struct Node
    {
        __int128 sum = 0;
        ll mx = -inf, mx2 = -inf, mn = inf, mn2 = inf, tag = 0;
        int cmx = 0, cmn = 0;
    };

    int n;
    vector<Node> t;

    SegmentBeats(const vector<ll> &a) : n(a.size()), t(4 * a.size() + 4)
    {
        assert(a.size() <= (INT_MAX - 4) / 4);
        if (n) build(1, 0, n, a);
    }
```

极值合并与递归建树（接上页同一结构体）：

```
    void pull(int p)
    {
        auto &u = t[p];
        const auto &a = t[p * 2], &b = t[p * 2 + 1];
        u.sum = a.sum + b.sum;
        u.mx = max(a.mx, b.mx);
        u.mx2 = max(a.mx == u.mx ? a.mx2 : a.mx,
                    b.mx == u.mx ? b.mx2 : b.mx);
        u.cmx = (a.mx == u.mx ? a.cmx : 0) + (b.mx == u.mx ? b.cmx : 0);
        u.mn = min(a.mn, b.mn);
        u.mn2 = min(a.mn == u.mn ? a.mn2 : a.mn,
                    b.mn == u.mn ? b.mn2 : b.mn);
        u.cmn = (a.mn == u.mn ? a.cmn : 0) + (b.mn == u.mn ? b.cmn : 0);
        u.tag = 0;
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(-inf < a[l] && a[l] < inf);
            t[p].sum = t[p].mx = t[p].mn = a[l];
            t[p].cmx = t[p].cmn = 1;
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

整段加法和单极值组截断（接上页同一结构体）：

```
    void apply_add(int p, int len, ll x)
    {
        auto &u = t[p];
        assert(-(__int128)inf < (__int128)u.mn + x &&
               (__int128)u.mx + x < inf);
        assert(-(__int128)inf < (__int128)u.tag + x &&
               (__int128)u.tag + x < inf);
        u.sum += (__int128)x * len;
        u.mx += x;
        u.mn += x;
        if (u.mx2 != -inf) u.mx2 += x;
        if (u.mn2 != inf) u.mn2 += x;
        u.tag += x;
    }

    // Only the maximum changes; equality with mx2 must descend instead.
    void apply_min(int p, ll x)
    {
        auto &u = t[p];
        if (u.mx <= x) return;
        assert(u.mx2 < x);
        u.sum += ((__int128)x - u.mx) * u.cmx;
        if (u.mn == u.mx) u.mn = x;
        else if (u.mn2 == u.mx) u.mn2 = x;
        u.mx = x;
    }

    void apply_max(int p, ll x)
    {
        auto &u = t[p];
        if (u.mn >= x) return;
        assert(x < u.mn2);
        u.sum += ((__int128)x - u.mn) * u.cmn;
        if (u.mx == u.mn) u.mx = x;
        else if (u.mx2 == u.mn) u.mx2 = x;
        u.mn = x;
    }
```

标记下传与有条件递归更新（接上页同一结构体）：

```
    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        if (t[p].tag)
        {
            apply_add(p * 2, m - l, t[p].tag);
            apply_add(p * 2 + 1, r - m, t[p].tag);
            t[p].tag = 0;
        }
        for (int c : {p * 2, p * 2 + 1})
        {
            apply_min(c, t[p].mx);
            apply_max(c, t[p].mn);
        }
    }

    // type: 0 = chmin, 1 = chmax, 2 = add.
    void update(int p, int l, int r, int ql, int qr, ll x, int type)
    {
        if (qr <= l || r <= ql) return;
        if (type == 0 && t[p].mx <= x) return;
        if (type == 1 && t[p].mn >= x) return;
        if (ql <= l && r <= qr)
        {
            if (type == 0 && t[p].mx2 < x)
            {
                apply_min(p, x);
                return;
            }
            if (type == 1 && x < t[p].mn2)
            {
                apply_max(p, x);
                return;
            }
            if (type == 2)
            {
                apply_add(p, r - l, x);
                return;
            }
        }
        push(p, l, r);
        int m = (l + r) / 2;
        update(p * 2, l, m, ql, qr, x, type);
        update(p * 2 + 1, m, r, ql, qr, x, type);
        pull(p);
    }
```

公开修改与区间和查询（接上页同一结构体）：

```
    void chmin(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 0);
    }

    void chmax(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 1);
    }

    void add(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 2);
    }

    __int128 query(int p, int l, int r, int ql, int qr)
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        return query(p * 2, l, m, ql, qr) + query(p * 2 + 1, m, r, ql, qr);
    }

    ll sum(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        __int128 ans = l < r ? query(1, 0, n, l, r) : 0;
        assert(LLONG_MIN <= ans && ans <= LLONG_MAX);
        return (ll)ans;
    }
};
```

模板题使用：Library Checker Range Chmin Chmax Add Range Sum

题意：维护数组，操作 0/1/2/3 分别为区间 chmin、chmax、加法、求和。全部使用 0-based 非空半开区间。N,Q≤200000；所有时刻元素绝对值 ≤10^12，修改参数绝对值 ≤2×10^12。因此延迟值满足本模板的范围要求，公开区间和也可用 long long 表示。输入区间不需要加减 1，按操作号调用对应方法。

所需模板：SegmentBeats（第 59 页，6.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_chmin_chmax_add_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    SegmentBeats t(a);
    while (q--)
    {
        int type, l, r;
        cin >> type >> l >> r;
        if (type == 3)
        {
            cout << t.sum(l, r) << '\n';
            continue;
        }
        long long x;
        cin >> x;
        if (type == 0) t.chmin(l, r, x);
        else if (type == 1) t.chmax(l, r, x);
        else t.add(l, r, x);
    }
}
// END USAGE
```

第七章　二叉搜索树 & 平衡树

7.1　二叉搜索树 & 平衡树

7.1.1　PBDS 有序树与顺序统计

功能： GNU pb_ds 红黑树实现的有序集合，在 `set` 的接口之外支持按排名访问：`find_by_order(k)` 取第 k 个（从 0 计）元素，`order_of_key(x)` 求严格小于 x 的元素个数。

核心思路： `tree_order_statistics_node_update` 让每个节点额外维护子树大小，排名查询沿根向下按左子树大小决定走向。集合不允许重复元素，需要可重集合时存 (值, 全局唯一编号) 二元组。

接口契约： GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp 和 ext/pb_ds/tree_policy.hpp。ost<T> 为去重有序集合；insert(x)、erase(x)、find(x)、lower_bound(x)、upper_bound(x) 沿用集合接口。order_of_key(x) 为严格小于 x 的元素数，find_by_order(k) 返回 0-based 第 k 个的迭代器，越界返回 end。默认 less<T>；改用其他严格弱序比较器时，排名和边界均按该比较器解释。单次 O(log n)，空间 O(n)，假设比较为 O(1)。重复值使用 ost<pair<T,int»，插入 {x,id} 且 id 全局唯一、1≤id<INT_MAX。order_of_key({x,0}) 统计小于 x，order_of_key({x,INT_MAX}) 统计不大于 x；find_by_order(k)->first 取值，须先检查 end。删除一个 x：it=lower_bound({x,0})，检查 it!=end 且 it->first==x 后 erase(it)。不可用 less_equal 模拟多重集合。前驱先检查迭代器是否为 begin，再递减；空集合和无解不得解引用 end。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 1 页。

序列中间插删、截取与版本共享的 GNU rope 见第 67 页。

```
template <class T, class C = less<T>>
using ost = __gnu_pbds::tree<T,
                             __gnu_pbds::null_type,
                             C,
                             __gnu_pbds::rb_tree_tag,
                             __gnu_pbds::tree_order_statistics_node_update>;
```

模板题使用：Library Checker ordered_set

题意：维护无重复整数集合，支持第 k 小、计数及含等号的前驱/后继。find_by_order 使用 0-based 排名；order_of_key 只计严格小于 x，故不大于 x 的数量需补上相等元素。越界查询按题意输出 -1。

所需模板：ost（第 65 页，7.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/ordered_set

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, x;
    cin >> n >> q;
    ost<int> s;
    while (n--)
    {
        cin >> x;
        s.insert(x);
    }
    while (q--)
    {
        int op;
        cin >> op >> x;
        if (op == 0) s.insert(x);
        if (op == 1) s.erase(x);
        if (op == 2)
        {
            auto it = s.find_by_order(x - 1);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
        if (op == 3) cout << s.order_of_key(x) + (s.find(x) != s.end()) << '\n';
        if (op == 4)
        {
            auto it = s.upper_bound(x);
            cout << (it == s.begin() ? -1 : *--it) << '\n';
        }
        if (op == 5)
        {
            auto it = s.lower_bound(x);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
    }
}
```

7.1.2 GNU rope 与版本共享

功能：GNU 扩展的 `rope`：支持在任意位置插入、删除、截取子串和按下标访问的序列；复制一个 rope 只共享根，修改副本不影响原序列，可用来保存多个历史版本。

核心思路：rope 用平衡二叉树把序列拼接成若干叶子块，节点带引用计数并共享子树。复制只增加根的引用；修改时复制受影响的路径与叶块（写时复制），其余部分继续共享，所以各版本互不干扰。

接口契约：实际类型为 __gnu_cxx::rope<T>，不是 std::rope；它是 GNU 扩展序列，包含 ext/rope。本工具链随后重新包含 cassert，保留断言接口。rp<int>、rp<char> 表示序列，默认空。size()、empty()、push_back(x)、insert(p,x)、erase(p,len)、replace(p,x)、substr(p,len) 直接沿用 rope；位置为 0-based，len 是长度而非右端点。约定 0≤p≤size，删除/截取要求 len≤size-p，读写单点要求 p<size。r[i] 读取第 i 项；substr 返回新序列。rp<T> b=a 共享底层树，随后通过容器接口修改 b 不影响 a，可用 vector<rp<int» 保存分支版本。复制构造共享根为 O(1)；随机访问沿树路径，正常平衡时 O(log n)。插入/删除/截取涉及路径、叶块复制和可能的再平衡，不能把每次操作都写成严格最坏 O(log n)；释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 O(n) 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值；函数生成型 rope 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本：Q 个零单元，复制根与 l/r，入队 replace(r++,x)，出队读取 [l++]；直接 push_back/erase 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB，具体失败与通过范围见 docs/ROPE-NOTES.md。适合需要序列中间插删、截取和少量分支快照的场景；只按下标读写通常使用 vector 更直接。需要区间和、翻转等懒标记聚合时使用隐式 Treap，rope 本身不提供这些接口。下方示例通过修改 b 演示 a 保持原版本。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 1 页。

```
template <class T> using rp = __gnu_cxx::rope<T>;
```

```
rp<int> a;
a.push_back(3);
a.insert(0, 7);
rp<int> b = a;
b.replace(1, 9);
b.erase(0, 1);
auto c = a.substr(0, 1);
int x = a[1];
```

模板题使用：Library Checker persistent_queue

题意：维护可持久化队列，每次从指定版本入队或出队并建立新版本。题目的 -1 号空版本映射为下标 0；rope 副本和头尾指针一起复制，replace 写入不会改变历史版本，出队只推进该版本队头。

所需模板：rp（第 67 页，7.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<rp<int>> s(q + 1);
    vector<int> l(q + 1), r(q + 1);
    s[0] = rp<int>(q, 0);
    for (int i = 1; i <= q; i++)
    {
        int op, t, x;
        cin >> op >> t;
        ++t;
        s[i] = s[t];
        l[i] = l[t];
        r[i] = r[t];
        if (op == 0)
        {
            cin >> x;
            s[i].replace(r[i]++, x);
        }
        else
        {
            cout << s[i][l[i]++] << '\n';
        }
    }
}
```

7.2 Treap

7.2.1 FHQ Treap：可重集合与顺序统计

功能：维护可重整数集合，支持插入、删除一个值、求排名、求第 k 小、求严格前驱和严格后继。

核心思路：无旋 Treap：节点按值有序、按随机优先级成堆。`split(p,x)` 把树分成值小于 x（`equal` 为真时为不超过 x）和其余两部分，`merge` 按优先级合并两棵值域有序的树。插入时按 x 分裂后把新节点夹在中间合并；删除时分成小于、等于、大于 x 三段，等于段去掉其根节点（合并它的左右子树）再拼回。排名、第 k 小只沿树下走并用子树大小计数，前驱、后继转化为按排名求第 k 小。

接口契约：insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在；rank(x) 为严格小于 x 的元素数加一，x 不必存在；kth(k) 要求 1≤k≤size()。prev/next 为严格前驱/后继，不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX，分裂不使用 x-1。随机优先级下单次期望 O(log n)，并非确定性最坏界。节点池不回收已删除节点，空间按累计插入次数计算；可在构造时指定随机种子。可合并平衡树：合并两棵树时把较小的一棵逐个 insert 进较大的一棵（启发式合并），每个元素至多移动 O(log n) 次，总 $O(n \log^2 n)$。

```
struct OrderedTreap
{
```

```
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0;
        unsigned long long pri = 0;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }

    int node(ll x)
    {
        int p = a.size();
        a.push_back({0, 0, 1, x, rng()});
        return p;
    }

    // left: val < x, or val <= x when equal is true. No x-1 overflow.
    void split(int p, ll x, int &l, int &r, bool equal = false)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        if (a[p].val < x || (equal && a[p].val == x))
        {
            l = p;
            split(a[p].r, x, a[p].r, r, equal);
        }
        else
        {
            r = p;
            split(a[p].l, x, l, a[p].l, equal);
        }
        pull(p);
    }

    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
```

```
        return r;
    }

    void insert(ll x)
    {
        int p = node(x), l, r;
        split(root, x, l, r);
        root = merge(merge(l, p), r);
    }

    bool erase(ll x)
    {
        int l, mid, r;
        split(root, x, l, mid);
        split(mid, x, mid, r, true);
        bool found = mid != 0;
        if (mid) mid = merge(a[mid].l, a[mid].r);
        root = merge(l, merge(mid, r));
        return found;
    }

    int size() const { return a[root].siz; }

    int less(ll x, bool equal = false) const
    {
        int p = root, ans = 0;
        while (p)
        {
            if (a[p].val < x || (equal && a[p].val == x))
            {
                ans += a[a[p].l].siz + 1;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return ans;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
        while (true)
        {
            int left = a[a[p].l].siz;
            if (k == left + 1) return a[p].val;
            if (k <= left)
                p = a[p].l;
            else
            {
                k -= left + 1;
                p = a[p].r;
            }
```

```
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedTreap（第 68 页，7.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedTreap g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

7.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和

功能：维护一个序列，支持在指定位置插入、删除一段、翻转一段的顺序、对一段加值、把一段 0/1 值逐位取反，以及查询一段的和。

核心思路：隐式 Treap：以子树大小作为下标，`split(p,k)` 分出前 k 个元素。每个区间操作都先分成前 $l-1$ 个、$[l,r]$、其余三段，对中段的根打标记后再合并回去。标记有三种：顺序翻转 `rev`（交换左右孩子）；值变换写成 $x \mapsto \pm x + \texttt{tag}$，取负时连同已有的 `tag` 一起取负，下传时先取负再加。位翻转即 $x \mapsto 1-x$，用一次取负加一次加 1 实现。`split/merge` 访问节点前先 `push`。

接口契约：insert(k,value) 在前 k 个元素后插入，0≤k≤size()；区间接口 erase、reverse、flip_bits、add、query 均使用 1-based 非空闭区间。reverse 反转元素顺序；flip_bits 要求区间内全为 0/1，逐位取反并保持顺序，不检查这一前提。query 返回和，二进制区间即 1 的个数。values() 递归中序返回完整序列，O(n)。其余操作期望 O(log n)，不是最坏界。符号与加法标记表示 s*x+b，位翻转通过取负后加 1 实现，下传必须先符号再加法；顺序反转另行维护。所有值、和、标记、乘法及取负中间结果须在 long long 范围内。删除不回收节点，空间按累计插入次数计算；辅助函数和节点数组只供内部使用。

```cpp
struct SequenceTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0, sum = 0, tag = 0;
        unsigned long long pri = 0;
        bool rev = false, neg = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply_add(int p, ll x)
    {
        if (!p) return;
        a[p].val += x;
        a[p].sum += x * a[p].siz;
        a[p].tag += x;
    }

    void apply_negate(int p)
    {
        if (!p) return;
        a[p].val = -a[p].val;
        a[p].sum = -a[p].sum;
        a[p].tag = -a[p].tag;
        a[p].neg = !a[p].neg;
    }
```

顺序反转、懒标记下传与分裂（接上页同一结构体）：

```
    void apply_reverse(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            apply_reverse(a[p].l);
            apply_reverse(a[p].r);
            a[p].rev = false;
        }
        if (a[p].neg)
        {
            apply_negate(a[p].l);
            apply_negate(a[p].r);
            a[p].neg = false;
        }
        if (a[p].tag)
        {
            apply_add(a[p].l, a[p].tag);
            apply_add(a[p].r, a[p].tag);
            a[p].tag = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入、删除与顺序反转（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after the first k elements, 0 <= k <= size().
    void insert(int k, ll value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 0, rng(), false, false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use 1-based inclusive intervals.
    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
    }

    void reverse(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_reverse(y);
        root = merge(merge(x, y), z);
    }
```

位翻转、加法、查询与输出（接上页同一结构体）：

```
    // Requires every value in [l,r] to be 0 or 1; preserves their order.
    void flip_bits(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_negate(y);
        apply_add(y, 1);
        root = merge(merge(x, y), z);
    }

    void add(int l, int r, ll value)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_add(y, value);
        root = merge(merge(x, y), z);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        ll ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<ll> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<ll> values()
    {
        vector<ll> answer;
        collect(root, answer);
        return answer;
    }
};
```

模板题使用：Luogu P3391

题意：初始序列为 1..n，多次翻转闭区间 [l,r]，最后输出序列。插入参数为前面已有元素数，翻转区间为 1-based 闭区间。

所需模板：SequenceTreap（第 72 页，7.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceTreap g;
    for (int i = 1; i <= n; i++) g.insert(i - 1, i);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        g.reverse(l, r);
    }
    for (long long x : g.values()) cout << x << ' ';
    cout << '\n';
}
```

7.2.3 隐式 Treap 位翻转与顺序反转

第 72 页 `flip_bits(l,r)` 逐位取反（要求区间元素均为 0/1），`reverse(l,r)` 反转顺序；均为 1-based 闭区间。二进制区间的 `query` 即 1 的个数。取反为 $x \mapsto 1-x$，用 $x \mapsto sx+b$（$s \in \{1,-1\}$）表示：取负时 $s \leftarrow -s, b \leftarrow -b$ 再加 1，下传**先符号、再加法**。顺序反转只交换儿子，与取负、加法可交换。所有标记与中间量须在 `long long` 内，不能对 `LLONG_MIN` 取负。逐次 `insert` 构建期望 $O(n \log n)$，单次操作期望 $O(\log n)$，不是最坏界。

7.3 Splay 树

7.3.1 Splay：可重集合与顺序统计

功能：维护可重整数集合，支持插入、删除一个值、求排名、求第 k 小、求严格前驱和严格后继。

核心思路：Splay 树，相同值存成一个节点并计数 `cnt`。每次操作沿二叉搜索树走到目标或最后访问的节点，再把它伸展到根，保证均摊对数。插入时值已存在就增计数，否则新建叶子。删除先 `find` 把目标伸展到根；计数多于 1 就减计数，否则断开左右子树，在左子树里把最大节点伸展为根，再把右子树接成它的右孩子。排名、第 k 小、前驱、后继都是一次带计数的下行查找。

接口契约：insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 O(log n)。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
struct OrderedSplay
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0, cnt = 0, siz = 0;
        ll val = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].siz; }

    void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }

    int node(ll value, int parent)
    {
        int x = a.size();
        a.push_back({{0, 0}, parent, 1, 1, value});
        return x;
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        else
            root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展与查找（接上页同一结构体）：

```
    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    // Splay the last visited node even when the value does not exist.
    int find(ll value)
    {
        int x = root, last = 0;
        while (x && a[x].val != value)
        {
            last = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
            splay(x);
        else if (last)
            splay(last);
        return x;
    }
```

插入与删除（接上页同一结构体）：

```
    void insert(ll value)
    {
        int x = root, parent = 0;
        while (x && a[x].val != value)
        {
            parent = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
        {
            ++a[x].cnt;
            pull(x);
        }
        else
        {
            x = node(value, parent);
            if (parent)
                a[parent].ch[value > a[parent].val] = x;
            else
                root = x;
        }
        splay(x);
    }

    bool erase(ll value)
    {
        int x = find(value);
        if (!x) return false;
        if (a[x].cnt > 1)
        {
            --a[x].cnt;
            pull(x);
            return true;
        }
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        if (!l)
        {
            root = r;
            return true;
        }
        root = l;
        x = l;
        while (a[x].ch[1]) x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = r;
        if (r) a[r].fa = x;
        pull(x);
        return true;
    }
```

排名与第 k 小（接上页同一结构体）：

```
    int rank(ll value)
    {
        int x = root, last = 0, ans = 1;
        while (x)
        {
            last = x;
            if (value <= a[x].val)
                x = a[x].ch[0];
            else
            {
                ans += a[a[x].ch[0]].siz + a[x].cnt;
                x = a[x].ch[1];
            }
        }
        if (last) splay(last);
        return ans;
    }

    ll kth(int k)
    {
        assert(1 <= k && k <= size());
        int x = root;
        while (true)
        {
            int left = a[a[x].ch[0]].siz;
            if (k <= left)
                x = a[x].ch[0];
            else if (k <= left + a[x].cnt)
            {
                splay(x);
                return a[x].val;
            }
            else
            {
                k -= left + a[x].cnt;
                x = a[x].ch[1];
            }
        }
    }
```

严格前驱与后继（接上页同一结构体）：

```
    optional<ll> prev(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val < value)
            {
                answer = x;
                x = a[x].ch[1];
            }
            else
                x = a[x].ch[0];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }

    optional<ll> next(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val > value)
            {
                answer = x;
                x = a[x].ch[0];
            }
            else
                x = a[x].ch[1];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedSplay（第 78 页，7.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedSplay g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

7.4 替罪羊树

7.4.1 替罪羊树：重复计数与墓碑清理

区分 size（副本总数）、nodes（物理节点数）、live（仍有副本的不同键数）；重复值增加计数，删空后保留墓碑。排名只累计副本数。回溯时若某孩子节点数占比超过 3/4，或有效键不足物理节点一半，就中序收集活节点按中点重建。树高 $O(\log(d+2))$（d 为不同键数），查询最坏对数，更新摊还对数，单次重建可能线性；空间按累计分配量计。排名定义为严格较小副本数加一（空树为 1）；严格前驱后继用 optional 表示缺失。

7.5 笛卡尔树

7.5.1 笛卡尔树与相等值优先级

功能：给定数组 a，构造它的笛卡尔树：按下标是二叉搜索树（中序遍历即原顺序），按值是小根堆，因此子树恰好对应一段区间，根是该区间的最小值。常用于把“区间最小值”问题转成树上问题。

核心思路：从左到右加入元素，单调栈 `st` 自底向上保存当前树的最右链，栈中值单调不降。加入 a_i 时弹出所有严格大于 a_i 的栈顶，最后弹出的那个（整条被弹链的顶端）成为 i 的左孩子；若栈非空，则 i 接为新栈顶的右孩子，替换其原右孩子，然后 i 入栈。相等值不弹出，故较早的相等元素留在上方。处理完后栈底即根。

接口契约：cartesian_tree(a) 返回 CartesianTree {root,left,right}，下标 0..n-1，缺失孩子及空树根为-1。中序遍历为原下标顺序，按值为小根堆；相等值保留较小下标在上方，即区间最左最小值为根。输入不修改，n 须能用 int 表示，元素比较须构成严格弱序，浮点 NaN 不适用。

比较 O(1) 时，构造 O(n)，结果数组及辅助单调栈均 O(n)。每点入栈一次、出栈至多一次；单调栈维护树的最右路径，不模拟 DFS。弹出节点中最后一个成为新节点的左孩子，剩余栈顶的右孩子改成新节点。P5854 用 1..n 排列，n≤10^7；P5854 已用 C++14 GCC9 满分通过，排名未核验。

```
struct CartesianTree
{
    int root;
    vector<int> left, right;
};

template <class T>
CartesianTree cartesian_tree(const vector<T> &a)
{
    int n = a.size();
    CartesianTree tree{-1, vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> st;
    st.reserve(n);
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!st.empty() && a[i] < a[st.back()])
        {
            last = st.back();
            st.pop_back();
        }
        tree.left[i] = last;
        if (!st.empty()) tree.right[st.back()] = i;
        st.push_back(i);
    }
    if (n) tree.root = st.front();
    return tree;
}
```

模板题使用：P5854 笛卡尔树

题意：输入 1..n 的排列，构造小根笛卡尔树并输出左右孩子的异或校验和。孩子由 0-based 转为 1-based，缺失孩子为 0；乘法使用 long long。

所需模板：CartesianTree（第 85 页，7.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5854

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto tree = cartesian_tree(a);
    long long left = 0;
    long long right = 0;
    for (int i = 0; i < n; i++)
    {
        left ^= 1LL * (i + 1) * (tree.left[i] + 2);
        right ^= 1LL * (i + 1) * (tree.right[i] + 2);
    }
    cout << left << ' ' << right << '\n';
}
```

第八章　可持久化数据结构

8.1　可持久化线段树

8.1.1　主席树：静态区间第 k 小

问题描述：给定不再修改的数组，多次询问区间 $[l,r]$ 内第 k 小的值（重复值按次数计）。

核心思路：值离散化后，对每个前缀 $a_1..a_i$ 建一棵权值线段树版本 `root[i]`，节点 `sum` 记该值域内的元素个数；版本 i 由版本 $i-1$ 复制一条根到叶的路径得到，其余节点共享。区间 $[l,r]$ 的值域计数等于版本 r 减版本 $l-1$。查询时两版本同步从根下走：左孩子计数差 $c \geq k$ 就进左子树，否则 $k \leftarrow k-c$ 进右子树，到叶即答案。

接口契约：构造传入 0-based 数组，kth(l,r,k) 的区间使用 1-based 闭区间；要求 1≤k≤r-l+1。离散化后每个前缀一个版本，单次 O(log n)，总空间 O(n log n)。树上路径第 k 小：每个点在父版本上插入点权，路径 (u,v) 用 root[u]+root[v]-root[lca]-root[fa[lca]] 四个版本同步二分。区间不同数个数：位置 i 在 pre[i]（上一次出现位置）处插入，查询 [l,r] 统计版本 r 减版本 l-1 中 pre<l 的个数。

```
struct PersistentKth
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    vector<long long> vals;

    PersistentKth(const vector<long long> &a) : vals(a)
    {
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (long long x : a)
        {
            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
        }
    }

    int add(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        ++t[p].sum;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int v = add(t[old].l, l, m, x);
                t[p].l = v;
            }
```

```
            else
            {
                int v = add(t[old].r, m + 1, r, x);
                t[p].r = v;
            }
        }
        return p;
    }

    long long kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
            if (k <= c)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
            {
                k -= c;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P3834

题意：给定不再修改的数组，查询闭区间 [l,r] 内第 k 小的值，重复值按出现次数计数。构造传入 0-based vector，查询 l、r、k 均从 1 开始。

所需模板：PersistentKth（第 87 页，8.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentKth tree(a);
    while (m--)
    {
        int l, r, k;
        std::cin >> l >> r >> k;
        std::cout << tree.kth(l, r, k) << '\n';
    }
}
```

8.1.2 可持久化区间加与版本拼接

功能：维护长度为 n 的数组的多个版本（初始全 0），支持在某版本上区间加得到新版本、把一个版本的某段替换成另一个版本同位置的值得到新版本，以及查询任一版本的区间和。

核心思路：可持久化线段树，按需复制节点，标记永久化：区间加只在完全覆盖的节点上累加 `lazy` 并修正 `sum`，标记从不下传；节点的真实区间和等于其 `sum` 加上所有祖先 `lazy` 之和乘区间长度，查询时用 `carry` 累计祖先标记。拼接 `replace` 同时走目标树 p 与源树 q，`offset` 累计“源祖先标记和减目标祖先标记和”；到完全覆盖处直接借用源子树 q，若 `offset` $\neq 0$ 则复制一份把 `offset` 补进它的 `lazy`，使挂到目标祖先下后数值不变。

接口契约：PersistentRange(n) 建立长度 n 的全零版本 0，n≥1；使用隐式节点，不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x；splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值，其他位置保留 dst；二者均返回新版本编号。query(v,l,r) 返回区间和，const 且不分配节点。下标 0-based、区间半开，允许空区间；空修改及零增量仍创建版本但复用根，整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先，用 offset 修正跨版本拼接；每次修改/拼接/查询 O(log n)，版本别名 O(1)，总节点 O(q log n)。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内；节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根，不要外部改写。

```
struct PersistentRange
{
    using ll = long long;

    struct Node
    {
        int left = 0, right = 0;
        ll sum = 0, lazy = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    explicit PersistentRange(int size) : n(size) { assert(n > 0); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return int(t.size()) - 1;
    }

    void pull(int p, int len)
    {
        t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
    }
```

区间加与版本拼接（接上页同一结构体）：

```
    int change(int p, int l, int r, int ql, int qr, ll value)
    {
        if (qr <= l || r <= ql) return p;
        int u = clone(p);
        if (ql <= l && r <= qr)
        {
            t[u].lazy += value;
            t[u].sum += value * (r - l);
            return u;
        }
        int m = l + (r - l) / 2;
        int a = change(t[p].left, l, m, ql, qr, value);
        int b = change(t[p].right, m, r, ql, qr, value);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }

    int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
    {
        if (qr <= l || r <= ql) return p;
        if (ql <= l && r <= qr)
        {
            if (offset == 0) return q;
            int u = clone(q);
            t[u].lazy += offset;
            t[u].sum += offset * (r - l);
            return u;
        }
        int u = clone(p), m = l + (r - l) / 2;
        offset += t[q].lazy - t[p].lazy;
        int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
        int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }
```

只读查询与对外接口（接上页同一结构体）：

```
    ll get(int p, int l, int r, int ql, int qr, ll carry) const
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
        carry += t[p].lazy;
        int m = l + (r - l) / 2;
        return get(t[p].left, l, m, ql, qr, carry) +
               get(t[p].right, m, r, ql, qr, carry);
    }

    void check(int version, int l, int r) const
    {
        assert(0 <= version && version < int(root.size()));
        assert(0 <= l && l <= r && r <= n);
    }

    int add(int version, int l, int r, ll value)
    {
        check(version, l, r);
        int p = l == r || value == 0 ? root[version]
                                     : change(root[version], 0, n, l, r, value);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    int splice(int destination, int source, int l, int r)
    {
        check(destination, l, r);
        check(source, l, r);
        int p = l == r ? root[destination]
                       : replace(root[destination], root[source], 0, n, l, r, 0);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    ll query(int version, int l, int r) const
    {
        check(version, l, r);
        return l == r ? 0 : get(root[version], 0, n, l, r, 0);
    }
};
```

8.1.3 可持久化并查集

功能：可持久化并查集：在任一历史版本上合并两个集合得到新版本，并能查询任一版本中两点是否连通、所在集合大小。

核心思路：并查集数组用可持久化线段树保存：叶子值非负表示父亲，负数表示该点是根且值为负的集合大小；空节点读出 -1，即初始全是大小为 1 的单点。因不能路径压缩，`merge` 按大小合并，把较小集合的根挂到较大集合的根下，树高为对数级；`find` 从 x 反复读父亲直到读到负数。一次合并只修改两个叶子（被挂根的父亲、新根的大小），各复制一条路径，作为新版本的根。

接口契约：PersistentDSU(n)，n>0，点号 0..n-1，版本 0 为单点集合。merge(ver,x,y) 从任意版本合并并返回新版本号，已连通也创建版本；copy(ver) 复制版本。find、same、size 只读，不压缩路径；按大小合并，等大时第一根

挂到第二根。区别于普通 dsu 的固定方向合并。稀疏可持久化线段树的叶子存父亲或负大小，空节点表示-1。初始化 O(1)，copy 均摊 O(1)，其余 O($\log^2$(n+1))；成功合并增加 O(log(n+1)) 节点。节点数/版本数须可用 int 索引。root、t 只读，get/set 内部函数勿直接调用；旧版本永不被改写。

```
struct PersistentDSU
{
    struct Node
    {
        int l = 0, r = 0, v = -1;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentDSU(int n) : n(n)
    {
        assert(n > 0);
    }

    int get(int p, int l, int r, int x) const
    {
        if (!p) return -1;
        if (r - l == 1) return t[p].v;
        int m = l + (r - l) / 2;
        if (x < m) return get(t[p].l, l, m, x);
        return get(t[p].r, m, r, x);
    }

    int set(int old, int l, int r, int x, int v)
    {
        Node node = t[old];
        int p = t.size();
        t.push_back(node);
        if (r - l == 1)
            t[p].v = v;
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = set(node.l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = set(node.r, m, r, x, v);
                t[p].r = q;
            }
        }
        return p;
    }
```

可持久化并查集（接上页同一结构体）：

```
    // Queries never compress paths or allocate nodes. Vertices are 0-based.
    int find(int ver, int x) const
    {
        assert(0 <= ver && ver < (int)root.size());
        assert(0 <= x && x < n);
        int p = get(root[ver], 0, n, x);
        while (p >= 0)
        {
            x = p;
            p = get(root[ver], 0, n, x);
        }
        return x;
    }

    int size(int ver, int x) const
    {
        return -get(root[ver], 0, n, find(ver, x));
    }

    bool same(int ver, int x, int y) const
    {
        return find(ver, x) == find(ver, y);
    }

    int copy(int ver)
    {
        assert(0 <= ver && ver < (int)root.size());
        root.push_back(root[ver]);
        return (int)root.size() - 1;
    }

    // Always create one version; ties attach the first root to the second.
    int merge(int ver, int x, int y)
    {
        x = find(ver, x);
        y = find(ver, y);
        if (x == y) return copy(ver);
        int a = get(root[ver], 0, n, x);
        int b = get(root[ver], 0, n, y);
        if (a < b)
        {
            swap(x, y);
            swap(a, b);
        }
        int p = set(root[ver], 0, n, x, y);
        p = set(p, 0, n, y, a + b);
        root.push_back(p);
        return (int)root.size() - 1;
    }
};
```

模板题使用：Luogu P3402 / 可持久化并查集

题意：n≤100000，m≤200000。1 a b 合并；2 k 回到第 k 次操作后的状态，可回到 0；3 a b 输出是否连通。每种操作均产生版本，操作号等于版本号，点号转 0-based，无异或。

所需模板：PersistentDSU（第 92 页，8.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentDSU d(n);
    for (int i = 1; i <= m; i++)
    {
        int op, x, y;
        cin >> op >> x;
        if (op == 2)
            d.copy(x);
        else
        {
            cin >> y;
            if (op == 1)
                d.merge(i - 1, x - 1, y - 1);
            else
            {
                cout << d.same(i - 1, x - 1, y - 1) << '\n';
                d.copy(i - 1);
            }
        }
    }
}
```

8.1.4 可持久化区间拼接的懒标记

代码见第 89 页。从全零版本开始，同一对象内版本共享坐标域；标记永久化：sum = left.sum + right.sum + lazy · len，查询只读。拼接把目标版本某半开区间替换为源版本同一区间；递归参数 offset 下降时更新为 offset + source.lazy − destination.lazy，完整覆盖时复制源节点并把 offset 加到 lazy 与 sum（offset 为零直接共享）。修改、拼接 $O(\log n)$ 新节点，查询 $O(\log n)$ 零新节点。不支持错位平移粘贴，不回收节点。和、差、标记与长度乘积须在 long long 内；节点数与版本数小于 `INT_MAX`。题目时间戳可回退时，用 ver[t] 映射时间戳到模板版本，不能把 add 返回的追加编号当时间戳。

8.1.5 杭州 2023 K：跨版本拼接应用

0-based，$F_i(r)$ 为 $[i,r]$ 剩余牌数，p 为 i 之后第一张与 a_i 相同的牌（无则 $p=n$）：

$$F_i(r)=\begin{cases}1+F_{i+1}(r), & r<p,\\ F_{p+1}(r), & r\ge p.\end{cases}$$

从 $i=n-1$ 向前：复制版本 $i+1$，$[i,p)$ 加 1，$[p,n)$ 拼接自版本 $p+1$。询问 $[l,r]$ 读版本 $l-1$ 的坐标 $r-1$；强制在线。构建 $O(n\log n)$，查询 $O(q\log n)$。

8.2 可持久化平衡树

8.2.1 可持久化有序 Treap：分叉版本与顺序统计

功能：维护可重整数集合的多个历史版本：在任一旧版本上插入或删除一个值得到新版本；在指定版本上查询元素个数、排名、第 k 小、严格前驱与后继。

核心思路：Treap 节点存值、重复次数 `cnt` 与子树大小 `siz`，修改时沿查找路径复制节点（`clone`），未改动的子树在版本间共享。插入递归到位置后，若新孩子优先级更高就把它旋转上来，只旋转新复制的节点。删除时若 `cnt` >1 只减计数，否则用 `merge` 合并左右子树代替该节点；值不存在时子树原样返回，不产生新节点。查询从 `root[v]` 沿树下走：`less` 统计小于（或不超过）x 的个数，前驱、后继转成按排名求 `kth`。

接口契约：版本 0 为空。insert(v,x)、erase(v,x)、copy(v) 追加并返回新版本；删除不存在的值也追加相同集合的版本。size/less/rank/kth/prev/next 均显式传版本且只读，不创建版本或树节点。重复值计数，rank 为严格小于 x 的副本数加一；kth 为 1-based 且要求合法，严格前后继缺失返回 nullopt。支持 long long 全范围，不用 x±1。随机优先级下期望 O(log(d+1))，d 为版本不同键数；copy 摊还 O(1)，没有最坏对数保证。递归复制更新路径，节点不回收，空间按全部历史分配节点与版本数计。节点/版本/各版副本数须小于 INT_MAX；t、root 及带节点编号的辅助函数属内部状态。当前常见 64 位布局每节点 32 字节；500000 次随机不同键插入实测约 1200 万节点，不预留的 vector 扩容可能超过 512MiB 峰值。预留 26*n+1 仅是明确预算而非上界，先核对内存限制；重新构造清空全部版本。

```
struct PersistentOrderedTreap
{
    using ll = long long;

    struct Node
    {
        ll val = 0;
        unsigned long long pri = 0;
        int l = 0, r = 0, siz = 0, cnt = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    mt19937_64 rng;

    PersistentOrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void check(int v) const { assert(0 <= v && v < (int)root.size()); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return (int)t.size() - 1;
    }

    void pull(int p) { t[p].siz = t[t[p].l].siz + t[t[p].r].siz + t[p].cnt; }
```

复制路径插入与旋转（接上页同一结构体）：

```
    // Only rotate freshly copied nodes; the promoted child was copied by add.
    int add(int p, ll x)
    {
        if (!p)
        {
            t.push_back({x, rng(), 0, 0, 1, 1});
            return (int)t.size() - 1;
        }
        int u = clone(p);
        if (x == t[u].val)
            t[u].cnt++;
        else if (x < t[u].val)
        {
            int q = add(t[u].l, x);
            t[u].l = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].l = t[q].r;
                t[q].r = u;
                pull(u);
                u = q;
            }
        }
        else
        {
            int q = add(t[u].r, x);
            t[u].r = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].r = t[q].l;
                t[q].l = u;
                pull(u);
                u = q;
            }
        }
        pull(u);
        return u;
    }
```

复制路径合并与删除（接上页同一结构体）：

```
68      int merge(int l, int r)
69      {
70          if (!l || !r) return l ? l : r;
71          if (t[l].pri > t[r].pri)
72          {
73              int u = clone(l);
74              int q = merge(t[u].r, r);
75              t[u].r = q;
76              pull(u);
77              return u;
78          }
79          int u = clone(r);
80          int q = merge(l, t[u].l);
81          t[u].l = q;
82          pull(u);
83          return u;
84      }
85
86      int remove(int p, ll x)
87      {
88          if (!p) return 0;
89          if (x == t[p].val)
90          {
91              if (t[p].cnt == 1) return merge(t[p].l, t[p].r);
92              int u = clone(p);
93              t[u].cnt--;
94              pull(u);
95              return u;
96          }
97          bool left = x < t[p].val;
98          int old = left ? t[p].l : t[p].r;
99          int q = remove(old, x);
100         if (q == old) return p;
101         int u = clone(p);
102         if (left)
103             t[u].l = q;
104         else
105             t[u].r = q;
106         pull(u);
107         return u;
108     }
```

版本接口、大小与排名（接上页同一结构体）：

```
    // Updates append versions; erase removes one occurrence, or changes nothing.
    int insert(int v, ll x)
    {
        check(v);
        int p = add(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int erase(int v, ll x)
    {
        check(v);
        int p = remove(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int v)
    {
        check(v);
        root.push_back(root[v]);
        return (int)root.size() - 1;
    }

    int size(int v) const
    {
        check(v);
        return t[root[v]].siz;
    }

    int less(int v, ll x, bool equal = false) const
    {
        check(v);
        int p = root[v], ans = 0;
        while (p)
        {
            if (t[p].val < x || (equal && t[p].val == x))
            {
                ans += t[t[p].l].siz + t[p].cnt;
                p = t[p].r;
            }
            else
                p = t[p].l;
        }
        return ans;
    }

    int rank(int v, ll x) const { return less(v, x) + 1; }
```

第 k 小与严格前驱后继（接上页同一结构体）：

```
    // Ranks and k are 1-based. No sentinel values are reserved.
    ll kth(int v, int k) const
    {
        assert(1 <= k && k <= size(v));
        int p = root[v];
        while (true)
        {
            int left = t[t[p].l].siz;
            if (k <= left)
                p = t[p].l;
            else if (k <= left + t[p].cnt)
                return t[p].val;
            else
            {
                k -= left + t[p].cnt;
                p = t[p].r;
            }
        }
    }

    optional<ll> prev(int v, ll x) const
    {
        int k = less(v, x);
        if (!k) return nullopt;
        return kth(v, k);
    }

    optional<ll> next(int v, ll x) const
    {
        int k = less(v, x, true);
        if (k == size(v)) return nullopt;
        return kth(v, k + 1);
    }
};
```

模板题使用：Luogu P3835 【模板】可持久化平衡树

题意：维护可重整数集合的历史版本；每条插入、删除或排名/第 k 小/严格前后继查询都从指定旧版本生成版本 i。删除缺失值忽略但仍产生版本；查询输出后 copy(v)，避免版本编号错位。第 k 小保证合法；缺失前后继分别输出-2147483647、2147483647，仅由驱动映射 optional。n≤500000、键绝对值 ≤10^9。reserve(26*n+1) 是约 32 字节节点的预留预算，不是分配上界；最大 n 预留约 397MiB，需先核对内存限制，不能将本地通过当作线上 AC。

所需模板：PersistentOrderedTreap（第 96 页，8.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3835

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    PersistentOrderedTreap tree;
    tree.root.reserve(n + 1);
    tree.t.reserve(26ULL * n + 1);
    for (int i = 1; i <= n; i++)
    {
        int v, op;
        long long x;
        cin >> v >> op >> x;
        if (op == 1)
            tree.insert(v, x);
        else if (op == 2)
            tree.erase(v, x);
        else
        {
            if (op == 3) cout << tree.rank(v, x) << '\n';
            if (op == 4) cout << tree.kth(v, (int)x) << '\n';
            if (op == 5) cout << tree.prev(v, x).value_or(-2147483647LL) << '\n';
            if (op == 6) cout << tree.next(v, x).value_or(2147483647LL) << '\n';
            tree.copy(v);
        }
    }
}
```

8.3 可持久化字典树

8.3.1 前缀可持久化 01 字典树

功能：按顺序追加若干个 bits 位非负整数，形成序列 $a_0, a_1, \ldots$；查询时给定区间 $[l, r)$ 与整数 x，求区间内某个 a_i 与 x 的最大异或值。

核心思路：可持久化 01 字典树：root[k] 是前 k 个数构成的字典树，每个节点的 cnt 记经过它的数的个数。追加 x 时从高位到低位复制旧版本的一条路径（共 bits + 1 个新节点），其余子树共享；节点 0 是自环的空节点。查询时从 root[r] 与 root[l] 同步下走，每位优先走与 x 该位相反的孩子，两版本在该孩子上 cnt 之差为正才说明区间内存在这样的数，否则只能走相同位。

接口契约：PersistentXorTrie(bits,cap)，1≤bits≤64，0≤cap≤(INT_MAX−1)/(bits+1)，构造时预留最多 cap 个键。append(x) 追加一个 uint64_t 原值，重复值按出现次数保留；满容量或 x 不满足位宽时返回 false 且逻辑状态不变。size() 为键数；max_xor(l,r,x) 从 0-based 半开区间 [l,r) 选择一个键 y，返回最大 x xor y，空区间 nullopt。查询 x 允许完整 uint64；不是子集异或、任意分支版本或字符串 Trie。构造 O(cap*bits) 容量空间，追加/查询 O(bits)；复制后首次追加为恢复预留空间可花 O(size*bits)。append 分配失败传播异常，逻辑内容不变；构造参数非法抛异常，查询下标是调用者前置条件，assert 只作调试检查。内部成员只读；不要复用已移动对象。

```
struct PersistentXorTrie
{
    using U = uint64_t;
    struct Node
    {
        int ch[2] = {}, cnt = 0;
    };
    int bits, cap;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentXorTrie(int bits, int cap) : bits(bits), cap(cap)
    {
        if (bits < 1 || bits > 64 || cap < 0)
            throw invalid_argument("PersistentXorTrie domain");
        if (cap > (INT_MAX - 1) / (bits + 1))
            throw length_error("PersistentXorTrie capacity");
        reserve();
    }

    void reserve()
    {
        t.reserve(1 + size_t(cap) * (bits + 1));
        root.reserve(size_t(cap) + 1);
    }

    int size() const { return (int)root.size() - 1; }
```

前缀追加与区间单值最大异或（接上页同一结构体）：

```
    bool append(U x)
    {
        if (size() == cap || (bits < 64 && (x >> bits))) return false;
        reserve(); // A copied vector need not preserve its original capacity.
        int old = root.back(), p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        root.push_back(p);
        t[p].cnt++;
        for (int d = bits - 1; d >= 0; d--)
        {
            int b = x >> d & 1, q = t.size();
            old = t[old].ch[b];
            copy = t[old];
            t.push_back(copy);
            t[p].ch[b] = q;
            p = q;
            t[p].cnt++;
        }
        return true;
    }

    optional<U> max_xor(int l, int r, U x) const
    {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return nullopt;
        int a = root[l], b = root[r];
        U y = 0;
        for (int d = bits - 1; d >= 0; d--)
        {
            int k = (x >> d & 1) ^ 1;
            if (t[t[b].ch[k]].cnt == t[t[a].ch[k]].cnt) k ^= 1;
            y |= U(k) << d;
            a = t[a].ch[k];
            b = t[b].ch[k];
        }
        return x ^ y;
    }
};
```

第九章　树套树

9.1　树状数组套权值线段树

9.1.1　动态区间第 k 小（树状数组套权值线段树）

问题描述：给定数组，支持单点修改为新值，以及询问区间 $[l,r]$ 内第 k 小的值。

核心思路：把所有初值和将来的新值一起离散化。按位置建树状数组，第 i 个节点挂一棵动态开点的权值线段树，记录树状数组覆盖的那段位置中每个值出现的次数。修改先在 $O(\log n)$ 棵树中删去旧值、再插入新值；计数变为 0 的节点挂到 `free_head` 空闲链上复用。查询时取出前缀 r 与前缀 $l-1$ 对应的两组树根，在值域上同步二分：左孩子计数之和（前一组减后一组）不少于 k 就整体往左走，否则减去后往右走。

接口契约：DynamicKth(a,future) 传入非空 0-based 初值，future 包含所有将来可能赋入的新值；内部自动合并初值并离散化。set(pos,value) 单点赋值，kth(l,r,k) 查询闭区间第 k 小，下标均为 1-based；1≤k≤r-l+1。允许重复值与完整 long long 权值；新值必须已在离散化集合内，操作可按原顺序执行，不保留历史版本。每个树状数组节点对应一棵可变权值线段树；删除旧值后插入新值。空节点回收到 free_head 链，重复修改不会按操作次数累计泄漏节点。设 D 为不同值数，S=n+future.size()，预处理 O(S log S+n log n log(D+1))，单次修改及查询 O(log n log(D+1))。节点数上界 B=(ceil(log2(D))+1) · $\sum$ lowbit(i)，求和 i=1..n；空间 O(S+n+B)。update、modify、id 和节点池均为内部接口；不能手动减去不存在的值。

```
struct DynamicKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, free_head = 0;
    vector<int> bit, rank;
    vector<ll> vals;
    vector<Node> t{Node{}};

    DynamicKth(const vector<ll> &a, vector<ll> future = {})
        : n(a.size()), bit(n + 1), rank(n + 1)
    {
        assert(n > 0);
        vals = future;
        vals.insert(vals.end(), a.begin(), a.end());
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (int i = 1; i <= n; i++)
        {
            rank[i] = id(a[i - 1]);
            modify(i, rank[i], 1);
        }
    }

    int id(ll value) const
    {
        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
```

```
33          assert(p < (int)vals.size() && vals[p] == value);
34          return p;
35      }
36
37      int new_node()
38      {
39          if (!free_head)
40          {
41              t.push_back(Node{});
42              return (int)t.size() - 1;
43          }
44          int p = free_head;
45          free_head = t[p].l;
46          t[p] = Node{};
47          return p;
48      }
49
50      int update(int p, int l, int r, int x, int delta)
51      {
52          if (!p) p = new_node();
53          t[p].sum += delta;
54          if (l < r)
55          {
56              int m = (l + r) / 2;
57              if (x <= m)
58              {
59                  int child = update(t[p].l, l, m, x, delta);
60                  t[p].l = child;
61              }
62              else
63              {
64                  int child = update(t[p].r, m + 1, r, x, delta);
65                  t[p].r = child;
66              }
67          }
68          if (!t[p].sum)
69          {
70              t[p].l = free_head;
71              free_head = p;
72              return 0;
73          }
74          return p;
75      }
76
77      void modify(int pos, int x, int delta)
78      {
79          for (int i = pos; i <= n; i += i & -i)
80              bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
81      }
```

```
    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        int x = id(value);
        if (rank[pos] == x) return;
        modify(pos, rank[pos], -1);
        modify(pos, x, 1);
        rank[pos] = x;
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
        vector<int> a, b;
        for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
        for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
        int L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int count = 0;
            for (int p : a) count += t[t[p].l].sum;
            for (int p : b) count -= t[t[p].l].sum;
            bool left = k <= count;
            if (!left) k -= count;
            for (int &p : a) p = left ? t[p].l : t[p].r;
            for (int &p : b) p = left ? t[p].l : t[p].r;
            int m = (L + R) / 2;
            if (left)
                R = m;
            else
                L = m + 1;
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P2617

题意：支持单点赋值及闭区间第 k 小。先读完操作，把所有修改的新值收集到 future，再构造并按原顺序执行；初值 vector 为 0-based，位置、查询区间和 k 为 1-based。Query 仅保存输入字段。

所需模板：DynamicKth（第 104 页，9.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    struct Query
    {
        char op;
        int x, y, k;
        long long value;
    };

    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n), future;
    for (auto &x : a) std::cin >> x;
    std::vector<Query> queries(m);
    for (auto &q : queries)
    {
        std::cin >> q.op;
        if (q.op == 'Q')
            std::cin >> q.x >> q.y >> q.k;
        else
        {
            std::cin >> q.x >> q.value;
            future.push_back(q.value);
        }
    }
    DynamicKth tree(a, future);
    for (const auto &q : queries)
    {
        if (q.op == 'Q')
            std::cout << tree.kth(q.x, q.y, q.k) << '\n';
        else
            tree.set(q.x, q.value);
    }
}
```

第十章　K-D Tree

10.1　动态二维 K-D Tree：单点累加与矩形和

功能：在线维护平面上的带权点集，支持给某个坐标的点累加权值（不存在则新建），以及查询闭矩形内所有点的权值和。

核心思路：二维 K-D Tree：按深度交替以 x、y 为划分轴（坐标相同时比较另一维），每个节点存自身点与权值，以及子树的包围盒、点数和权值和。插入沿划分方向下行，相同坐标直接累加；回溯时若某个孩子的点数超过子树的 3/4，就中序收集该子树所有节点，用 `nth_element` 取中位数递归重建（替罪羊式重构），保持树高对数级。查询时包围盒与矩形不相交就剪掉，被矩形完全包含就直接返回子树和，否则计入本节点的点后递归两个孩子。

接口契约：KDTreeSum<Coord=int>(reserve_points=0) 初始为空；Coord 取 int 或 long long。add(x,y,delta) 累加有符号 long long 权值，相同坐标只占一个节点；零权节点仍保留，不支持删除坐标。query(x1,y1,x2,y2) 查询闭矩形和，左下端超过右上端则返回 0。坐标只比较，不相减，不用有限哨兵；可用所选整数类型极值。当前各点绝对权值总和及更新中间量须在 long long 内。不同点数 d、预留容量和节点编号须小于 INT_MAX。按轴交替分割，子树过重时递归收集编号并原位重建；树高 O(log(d+1))。nth_element 按平均线性选择计，插入摊还 $O(\log^2(d+1))$，单次大重建可 O(d log(d+1))；矩形查询最坏 O(d)，不能保证对数。空间 O(d)，预留容量另计，默认 int 节点常见为 56 字节；紧内存题预留最大不同点数，避免扩容峰值。a/root 及带节点编号的函数均为内部状态；可重新赋值空对象重置。

```
template <class Coord = int> struct KDTreeSum
{
    using ll = long long;
    using Point = array<Coord, 2>;

    struct Node
    {
        Point point{}, low{}, high{};
        array<int, 2> child{};
        int size = 1;
        ll value = 0, sum = 0;
    };

    vector<Node> a;
    int root = 0;

    KDTreeSum(int reserve_points = 0)
    {
        a.reserve(reserve_points + 1);
        a.push_back(Node{});
        a[0].size = 0;
    }

    bool less(Point x, Point y, int axis) const
    {
        if (x[axis] != y[axis]) return x[axis] < y[axis];
        return x[axis ^ 1] < y[axis ^ 1];
    }
```

包围盒维护、递归收集与重建（接上页同一结构体）：

```
    void pull(int p)
    {
        a[p].low = a[p].high = a[p].point;
        a[p].size = 1;
        a[p].sum = a[p].value;
        for (int q : a[p].child)
        {
            if (!q) continue;
            a[p].size += a[q].size;
            a[p].sum += a[q].sum;
            for (int d = 0; d < 2; d++)
            {
                a[p].low[d] = min(a[p].low[d], a[q].low[d]);
                a[p].high[d] = max(a[p].high[d], a[q].high[d]);
            }
        }
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].child[0], v);
        v.push_back(p);
        collect(a[p].child[1], v);
    }

    int build(vector<int> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l,
                    v.begin() + m,
                    v.begin() + r,
                    [&](int x, int y)
                    {
                        return less(a[x].point, a[y].point, axis);
                    });
        int p = v[m];
        a[p].child[0] = build(v, l, m, axis ^ 1);
        a[p].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(p);
        return p;
    }
```

点权累加与闭矩形查询（接上页同一结构体）：

```
    int add(int p, Point point, ll delta, int axis)
    {
        if (!p)
        {
            p = a.size();
            a.push_back(Node{});
            a[p].point = a[p].low = a[p].high = point;
            a[p].value = a[p].sum = delta;
            return p;
        }
        if (a[p].point == point)
            a[p].value += delta;
        else
        {
            int side = !less(point, a[p].point, axis);
            a[p].child[side] = add(a[p].child[side], point, delta, axis ^ 1);
        }
        pull(p);
        int heavy = max(a[a[p].child[0]].size, a[a[p].child[1]].size);
        if (4LL * heavy <= 3LL * a[p].size) return p;
        vector<int> v;
        v.reserve(a[p].size);
        collect(p, v);
        return build(v, 0, v.size(), axis);
    }

    void add(Coord x, Coord y, ll delta)
    {
        root = add(root, {x, y}, delta, 0);
    }

    ll query(int p, Point low, Point high) const
    {
        if (!p) return 0;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[p].high[d] < low[d] || high[d] < a[p].low[d]) return 0;
            covered &= low[d] <= a[p].low[d] && a[p].high[d] <= high[d];
            inside &= low[d] <= a[p].point[d] && a[p].point[d] <= high[d];
        }
        if (covered) return a[p].sum;
        ll answer = inside ? a[p].value : 0;
        for (int q : a[p].child) answer += query(q, low, high);
        return answer;
    }

    ll query(Coord x1, Coord y1, Coord x2, Coord y2) const
    {
        if (x1 > x2 || y1 > y2) return 0;
        return query(root, {x1, y1}, {x2, y2});
    }
};
```

10.2 静态 K-D Tree 最近与最远点

功能：给定静态点集，查询离给定点最近（或最远）的 k 个点的编号，按欧氏距离排序，等距时编号小者在前。

核心思路：按深度轮换坐标轴，以中位数递归建树，每个节点记录子树包围盒与子树中的最小编号 `first`。查询维护大小至多 k 的大根堆，键为 (平方距离, 编号)；求最远时用负的距离，统一成“越小越好”。`bound` 给出子树内可能达到的最好键：最近时是查询点到包围盒的距离，最远时是到包围盒最远角的距离取负，编号取 `first`。堆满且该下界不优于堆顶就整棵剪掉；否则用当前点更新堆，并先搜下界更好的孩子。最后把堆依次弹出再反转。

接口契约：KDNearest<D>(points)，D>0，Point=array<int,D>，坐标为有符号 32 位整数，输入编号 0..n-1 且允许重合点。query(q,k,far=false) 返回最近 k 个原编号；far=true 改为最远 k 个。结果按距离优先、等距编号小者优先排列，0≤k≤n；k=0 返回空，包含与查询重合的点，不自动排除自身。平方距离用 128 位整数精确计算。p 保留输入顺序；p/a/root 仅供读取，重建请重新赋值。递归中位数建树，平均 O(Dn log(n+1))，空间 O(Dn)；包围盒剪枝没有对任意数据的次线性保证，单次最坏 O(Dn+n log(k+1))，额外 O(k+log(n+1)) 空间。search/build/bound 为内部辅助，不直接调用。

```
template <int D = 2> struct KDNearest
{
    static_assert(D > 0);
    using Point = array<int, D>;
    using Wide = __int128_t;
    using Key = pair<Wide, int>;

    struct Node
    {
        Point low, high;
        array<int, 2> child{-1, -1};
        int first;
    };

    vector<Point> p;
    vector<Node> a;
    int root;

    KDNearest(const vector<Point> &points = {}) : p(points), a(p.size())
    {
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }
```

中位数建树与距离界（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int axis)
    {
        if (l == r) return -1;
        int m = (l + r) / 2;
        nth_element(order.begin() + l, order.begin() + m, order.begin() + r,
            [&](int x, int y)
            {
                return pair(p[x][axis], x) < pair(p[y][axis], y);
            });
        int u = order[m];
        a[u].low = a[u].high = p[u];
        a[u].first = u;
        a[u].child[0] = build(order, l, m, (axis + 1) % D);
        a[u].child[1] = build(order, m + 1, r, (axis + 1) % D);
        for (int v : a[u].child)
        {
            if (v == -1) continue;
            a[u].first = min(a[u].first, a[v].first);
            for (int d = 0; d < D; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
        return u;
    }

    Wide distance(const Point &x, const Point &y) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide t = Wide(x[d]) - y[d];
            ans += t * t;
        }
        return ans;
    }

    Key bound(int u, const Point &q, bool far) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide l = Wide(a[u].low[d]) - q[d];
            Wide r = Wide(a[u].high[d]) - q[d];
            if (far) ans -= max(l * l, r * r);
            else if (l > 0) ans += l * l;
            else if (r < 0) ans += r * r;
        }
        return {ans, a[u].first};
    }
```

剪枝搜索与查询入口（接上页同一结构体）：

```
    void search(int u, const Point &q, int k, bool far,
                priority_queue<Key> &heap) const
    {
        if (u == -1) return;
        if ((int)heap.size() == k && bound(u, q, far) >= heap.top()) return;
        Wide dist = distance(p[u], q);
        Key cur{far ? -dist : dist, u};
        if ((int)heap.size() < k || cur < heap.top())
        {
            heap.push(cur);
            if ((int)heap.size() > k) heap.pop();
        }
        int x = a[u].child[0];
        int y = a[u].child[1];
        if (x == -1 || (y != -1 && bound(y, q, far) < bound(x, q, far)))
        {
            swap(x, y);
        }
        search(x, q, k, far, heap);
        search(y, q, k, far, heap);
    }

    vector<int> query(const Point &q, int k, bool far = false) const
    {
        assert(0 <= k && k <= (int)p.size());
        if (k == 0) return {};
        priority_queue<Key> heap;
        search(root, q, k, far, heap);
        vector<int> ans;
        while (!heap.empty())
        {
            ans.push_back(heap.top().second);
            heap.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

10.3 动态 K-D Tree：矩形和与重建边界

KDTreeSum 按深度交替坐标轴，重复坐标累加权值，孩子过重（超过 3/4）时 nth_element 重建。树高 $O(\log(d+1))$，矩形查询最坏 $O(d)$；插入摊还 $O(\log^2(d+1))$。矩形闭边界；绝对权值和及中间量须在 long long 内。零权保留节点，负权不影响剪枝。

第十一章　动态树

11.1　Link Cut Tree

11.1.1　Link-cut tree：动态森林与路径异或

功能：维护带点权的动态森林，支持连边、删边、修改点权、判断两点是否连通，以及查询两点间路径上点权的异或和。

核心思路：把每棵树剖成若干条实链，每条实链用一棵按深度为序的 Splay 维护，Splay 根的 `fa` 指向链顶在原树中的父亲（虚边），`is_root` 据此判断。`access(x)` 自下而上把根到 x 的路径变成一条实链；`make_root` 再整体打翻转标记使 x 成为原树根。路径查询先把 x 设为根再 `access(y)`，y 所在 Splay 的 `sum` 即答案。`splay` 前用显式栈自上而下下传翻转标记。

思路中的情况	代码中的处理	行
把根到 x 的路径变成实链	逐段 `splay`，把上一段接为右孩子，原右孩子变为虚边	77
换根	`access` 后翻转整条链，深度顺序颠倒	92
找所在树的根	`access` 后一路走左孩子找到深度最小点并伸展	98
连边	x 设为根，若 y 所在树根不是 x 就把 x 的父亲设为 y	116
删边	x 为根、`access(y)` 后，路径恰为 x, y 两点才是真实的边	125
路径查询	x 为根、`access(y)` 后读 y 的 `sum`	144

接口契约：构造 LinkCutTree(n)，点号 1..n，初始权值为零。set(x,value) 修改点权；link(x,y) 仅在不同连通块之间连边并返回是否成功；cut(x,y) 只删除真实存在的边，否则返回 false。connected 查询连通性；query(x,y) 返回路径点权异或，要求两点连通，支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 O(log n)，O(n) 空间，splay 下传使用显式栈，支持长链。权值为 unsigned long long。仅维护路径异或，不包含虚子树聚合或动态直径。路径最值或路径和：把异或聚合换成对应运算（如 max），在 pull 中合并左右子树与自身；路径加等标记须在 splay 前沿路径下传，与翻转标记一起在 push 中下传。

```
struct LinkCutTree
{
    using U = unsigned long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        U val = 0, sum = 0;
        bool rev = false;
    };

    vector<Node> a;
    vector<int> st;

    LinkCutTree(int n) : a(n + 1), st(n + 1) {}

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }
```

```
40      void rotate(int x)
41      {
42          int y = a[x].fa, z = a[y].fa;
43          int k = a[y].ch[1] == x;
44          int w = a[x].ch[k ^ 1];
45          if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46          a[x].fa = z;
47          a[y].ch[k] = w;
48          if (w) a[w].fa = y;
49          a[x].ch[k ^ 1] = y;
50          a[y].fa = x;
51          pull(y);
52          pull(x);
53      }
54
55      void splay(int x)
56      {
57          int top = 0, y = x;
58          st[top++] = y;
59          while (!is_root(y))
60          {
61              y = a[y].fa;
62              st[top++] = y;
63          }
64          while (top) push(st[--top]);
65          while (!is_root(x))
66          {
67              int y = a[x].fa, z = a[y].fa;
68              if (!is_root(y))
69              {
70                  bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71                  rotate(same ? y : x);
72              }
73              rotate(x);
74          }
75      }
```

```
77      void access(int x)
78      {
79          int last = 0;
80          for (int y = x; y; y = a[y].fa)
81          {
82              splay(y);
83              // The old right child's fa remains y: it becomes a path-parent.
84              a[y].ch[1] = last;
85              if (last) a[last].fa = y;
86              pull(y);
87              last = y;
88          }
89          splay(x);
90      }
91
92      void make_root(int x)
93      {
94          access(x);
95          reverse(x);
96      }
97
98      int find_root(int x)
99      {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }
```

```
    bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x) return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        make_root(x);
        access(y);
        if (a[y].ch[0] != x || a[x].ch[1]) return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    void set(int x, U value)
    {
        access(x);
        a[x].val = value;
        pull(x);
    }

    // x and y must be connected. This is a path aggregate, not a subtree sum.
    U query(int x, int y)
    {
        make_root(x);
        access(y);
        return a[y].sum;
    }
};
```

模板题使用：Luogu P3690

题意：维护点权森林，支持路径异或、连边、删边和单点赋值。初始没有边；0 查询、1 连边、2 删边、3 赋值。题目保证查询两点连通，成环的连边和不存在的删边由接口拒绝。点号 1..n。

所需模板：LinkCutTree（第 114 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3690

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LinkCutTree g(n);
    for (int i = 1; i <= n; i++)
    {
        unsigned long long x;
        cin >> x;
        g.set(i, x);
    }
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 0)
            cout << g.query(x, y) << '\n';
        else if (op == 1)
            g.link(x, y);
        else if (op == 2)
            g.cut(x, y);
        else
            g.set(x, y);
    }
}
```

11.1.2 动态树路径乘积：虚子树与双向信息

$$F(u)=\sum_{v\in V}\prod_{x\in \mathrm{path}(u,v)} w_x,$$

含单点路径（孤立点答案为点权）。固定根时 $F(u)=w_u(1+\sum_{v\text{ 为子节点}}F(v))$。LCT 辅助树按实链维护链积 P、自链首的贡献 L、自链尾的贡献 R，每点另记虚儿子贡献和 G：

$$P=P_l w P_r,\qquad L=L_l+P_l w(1+G+L_r),\qquad R=R_r+P_r w(1+G+R_l).$$

空树 $P=1, L=R=0$。翻转须同时交换左右儿子与 L,R。access 时旧右链的 L 加入 G，新右链的 L 从 G 减去。修改点权前先换为根；换根后根处的 L 即答案。只用加减乘，零权、负权与合数模数均适用；固定树，不含连边删边。修改和查询摊还 $O(\log n)$，空间 $O(n)$；递归最坏深度 $O(n)$。

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　动态模整数与合数模逆元

功能：运行时设定模数的取模整数，模数不要求是素数。支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。不同 `tag` 各自持有独立的模数。

核心思路：成员 `v` 始终是 $[0,p)$ 内的代表，加减只做一次条件校正。乘法用 Barrett 约减代替 64 位除法：`set_mod` 预存 $\mathtt{im}\approx\lceil 2^{64}/p\rceil$，对乘积 z 用 $\lfloor z\cdot \mathtt{im}/2^{64}\rfloor$ 估计商 q，q 至多偏大 1，因此 $z-qp$ 为负时补一个 p。求逆对 (v,p) 做迭代扩展欧几里得并维护 v 的系数，最后余数为 1 才可逆，系数即逆元。

接口契约：mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline unsigned long long im = -1ULL / 998244353 + 1; // Barrett reciprocal
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        im = -1ULL / p + 1;
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        unsigned long long z = 1ULL * v * b.v, y = (unsigned long long)((__uint128_t(z) * im) >> 64) * mod;
        r.v = int(z - y + (z < y ? mod : 0));
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；求前缀积后只对总积求一次逆（try_inv），倒序乘出各项逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：mint（第 121 页，A.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    // Batch inverse: one inversion of the prefix product.
    vector<mint<>> prefix(n + 1, 1);
    for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] * a[i];
    auto total = prefix[n].try_inv();
    if (!total) return 1;
    mint<> suffix = *total, answer = 0;
    vector<mint<>> inverse(n);
    for (int i = n - 1; i >= 0; i--)
    {
        inverse[i] = suffix * prefix[i];
        suffix *= a[i];
    }
    mint<> weight = 1;
    for (auto x : inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

A.1.2 静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

算法索引

CartesianTree, 85
cc_hash_table, 1

dsu, 3
DynamicKth, 104

Fenwick, 23

gp_hash_table, 1
gp_map, 1

KDNearest, 111
KDTreeSum, 108

lazy_segtree, 28
LCT
    虚子树, 119
LeftistHeap, 15
LiChao, 56
LinkCutTree, 114

MergeSplitTree, 42
mint, 121
ModInt, 125
MonotoneStackSeg, 39

offline_rectangle_sum, 24
OrderedSplay, 78
OrderedTreap, 68
ost, 65

PersistentDSU, 92
PersistentKth, 87
PersistentOrderedTreap, 96
PersistentRange, 89
PersistentXorTrie, 102
pheap, 13
PotentialDSU, 7

RectangleFenwick, 21
RollbackDSU, 5
rp, 67

SegmentBeats, 59
SegmentLiChao, 51
segtree, 34
SequenceTreap, 72

StaticRMQ, 17

TimeConnectivity, 9
Treap
    位翻转, 77

动态树 DP, 119

替罪羊树, 84